



Rapport Mini-projet

Module : Big Data

Titre : Système distribué d'analyse et de détection de fraude transactionnelle en temps réel basé sur le Machine Learning et le Federated Learning

Réalisé par :

AGUELLOUL Mouad

Encadré par :

Pr. CHAKOUR Iman

Soutenu le 2026

Résumé

Avec l'augmentation des transactions numériques, les systèmes financiers doivent faire face à des tentatives de fraude de plus en plus fréquentes et complexes. Les solutions classiques deviennent limitées lorsqu'il s'agit de traiter des flux de données massifs et temps réel tout en préservant la confidentialité des données des agences.

Le projet FlowGuard propose une solution intelligente de détection de fraude en temps réel basée sur les technologies Big Data, le Machine Learning et le Federated Learning. L'objectif principal est de détecter automatiquement les transactions suspectes tout en permettant à plusieurs agences de collaborer sans partager directement leurs données sensibles.

Le système utilise des streams simulant des transactions financières provenant de différentes agences. Ces données sont transmises via Apache Kafka, puis nettoyées et validées avant leur traitement. Un pipeline de Machine Learning calcule ensuite un score de fraude pour chaque transaction à l'aide de modèles locaux supervisés. Lorsqu'un score dépasse un seuil défini, une alerte est générée automatiquement.

FlowGuard intègre également un mécanisme de Federated Learning basé sur l'algorithme FedAvg. Chaque agence entraîne son propre modèle local et partage uniquement les poids du modèle avec le serveur central, qui construit ensuite un modèle global plus performant.

Les données, alertes et modèles sont stockés dans Neo4j afin d'assurer le suivi et l'analyse des informations. Enfin, un dashboard développé avec Dash/Plotly permet de superviser le système en temps réel grâce à des graphiques, indicateurs et tableaux interactifs.

Ce projet combine ainsi traitement temps réel, apprentissage distribué et visualisation interactive afin de proposer une solution moderne de détection de fraude financière.

Table de matières

Table des figures	4
Introduction générale	5
Chapitre 2 : Contexte et analyse	8
Introduction du chapitre.....	8
2.2. Présentation du projet et contexte.....	8
2.3. Cahier des charges	9
2.3.1. Traitement des flux en temps réel	9
2.3.2. Nettoyage et validation des données	9
2.3.3. Extraction de caractéristiques	10
2.3.4. Détection locale de fraude.....	10
2.3.5. Apprentissage fédéré	10
2.3.6. Stockage et traçabilité	10
2.3.7. Supervision visuelle.....	11
2.3.8. Modularité et maintenabilité.....	11
Conclusion du chapitre	11
Chapitre 3 : Modélisation	12
Introduction du chapitre.....	12
3.2. Diagramme de cas d'utilisation	12
3.3. Diagramme de séquence : Streams et nettoyage	15
3.4. Diagramme de séquence : Pipeline ML	17
3.4. Diagramme de séquence : Federated Learning.....	19
3.4. Diagramme de séquence : Dashboard temps réel	20
3.4. Diagramme de déploiement.....	23
Conclusion du chapitre	24
Chapitre 4 : Outils technologiques et réalisation	25
Introduction du chapitre.....	25
4.2. Outils technologiques	25
4.2.1. Outils de développement	25
4.2.2. Outils de conception	29

4.3. Réalisation	30
4.3.1. Architecture générale du système.....	30
4.3.2. Mise en place de l'environnement de développement	30
4.3.3. Configuration d'Apache Kafka et des topics.....	32
4.3.4. Génération et nettoyage des transactions simulées (Streams)	33
4.3.5. Réalisation du pipeline de Machine Learning	35
4.3.6. Gestion des modèles locaux et réentraînement.....	36
4.3.7. Implémentation du Federated Learning avec FedAvg	37
4.3.8. Intégration de Neo4j et stockage des données	37
4.3.9. Développement du dashboard temps réel	39
4.3.10. Présentation des interfaces de l'application	40
Conclusion du chapitre	52
Conclusion et perspectives	53
Webographie.....	56

Table des figures

Figure 1 : Diagramme de cas d'utilisation	14
Figure 2 : Diagramme de séquence Streams et nettoyage	16
Figure 3 : Diagramme de séquence Pipeline ML	18
Figure 4 : Diagramme de séquence Federated Learning	20
Figure 5 : Diagramme de séquence Dashboard temps réel	22
Figure 6 : Diagramme de déploiement	23
Figure 7 : Python logo	25
Figure 8 : Kafka logo	26
Figure 9 : Scikit-learn logo	26
Figure 10 : Neo4j logo	27
Figure 11 : Dash - Plotly logo	27
Figure 12 : NumPy logo	28
Figure 13 : Logo UML	29
Figure 14 : Logo PlantUML	29
Figure 15 : Architecture Kafka	33
Figure 16 : Shema neo4j	39
Figure 17 : Overview_1	41
Figure 18 : Overview_2	42
Figure 19 : Alerts_1	43
Figure 20 : Alerts_2	44
Figure 21 : ML_1	46
Figure 22 : ML_2	47
Figure 23 : FL_1	48
Figure 24 : FL_2	49
Figure 25 : Transactions_1	50
Figure 26 : Transactions_2	51
Figure 27 : Transactions_3	52

Introduction générale

Avec la généralisation des services financiers numériques et l'augmentation continue des échanges dématérialisés, la détection de fraude est devenue un enjeu majeur pour les organismes de transfert d'argent, les institutions financières et, plus largement, pour l'ensemble des systèmes transactionnels. Dans un contexte où les opérations sont exécutées en temps réel, à grande vitesse et sur des volumes de données élevés, les méthodes de contrôle classiques deviennent souvent insuffisantes pour repérer rapidement les comportements suspects. Les fraudes peuvent en effet se manifester sous des formes variées : montants atypiques, transactions répétées, exécution à des heures inhabituelles, localisation incohérente, usage étranger ou en ligne, ou encore enchaînement anormal d'opérations. La difficulté ne réside donc pas seulement dans l'identification d'un acte frauduleux, mais aussi dans la capacité à le détecter assez tôt pour limiter les pertes financières, réduire les risques opérationnels et renforcer la confiance dans le système.

L'étude de cette problématique prend une importance particulière dans le cadre des agences de transfert d'argent, où les transactions doivent être traitées rapidement tout en restant contrôlées avec rigueur. Dans ce type d'environnement, la diversité des profils clients et des habitudes de consommation rend l'analyse encore plus complexe. En effet, un même comportement peut être normal pour une agence et anormal pour une autre, selon le canal utilisé, les montants habituels, les lieux de transaction ou la saisonnalité de l'activité. C'est précisément ce caractère hétérogène qui justifie l'intérêt d'une approche intelligente, distribuée et adaptée au temps réel. Le projet FlowGuard s'inscrit dans cette logique : il vise à simuler un système réaliste de détection de fraude pour trois agences distinctes, chacune présentant un comportement transactionnel spécifique, afin de reproduire des situations proches d'un déploiement industriel.

Dans ce contexte, la problématique étudiée peut être formulée ainsi : comment concevoir une chaîne complète de détection de fraude capable de traiter des transactions en continu, de les nettoyer, d'en extraire des caractéristiques pertinentes, de les évaluer par des modèles locaux adaptés à chaque agence, puis d'agréger ces modèles dans une logique d'apprentissage fédéré, tout en assurant la

traçabilité et la visualisation des événements ? Cette question est centrale, car elle implique à la fois des contraintes de performance, de fiabilité, de modularité et d'évolutivité. Une solution pertinente doit être capable de gérer l'arrivée continue des données, de s'adapter à l'évolution des comportements, et de fournir des résultats exploitables en temps réel. FlowGuard répond à ces exigences par une architecture orientée pipeline, structurée autour de Kafka pour la diffusion des flux, d'un module de nettoyage, d'un module de feature engineering, d'un apprentissage automatique local, d'un agrégateur fédéré et d'un stockage graphique dans Neo4j.

Le présent projet apporte plusieurs contributions concrètes. Tout d'abord, il met en place une simulation complète de flux transactionnels via trois générateurs correspondant à trois agences, avec des comportements distincts et des profils de fraude différenciés. Cette simulation permet d'alimenter un pipeline temps réel réaliste et de tester le système dans des conditions proches du terrain. Ensuite, les transactions brutes sont soumises à une phase de nettoyage et de validation, afin de garantir la cohérence des données avant leur exploitation par les modèles. Cette étape est essentielle, car la qualité de la détection dépend directement de la qualité des données d'entrée.

Le projet introduit également un module de représentation des données fondé sur 20 caractéristiques numériques, conçu pour transformer chaque transaction en vecteur exploitable par un modèle de classification. Cette étape de feature engineering assure une dimension fixe, sans valeurs non définies, et permet d'intégrer des informations pertinentes telles que le montant, l'heure, le jour, l'usage en ligne ou à l'étranger, ainsi que des variables catégorielles relatives aux marchands et aux localisations. Sur cette base, chaque agence dispose d'un modèle local entraîné avec un apprentissage incrémental, fondé sur SGDClassifier avec une fonction de perte logistique, compatible avec un schéma d'agrégation fédérée. Le système prévoit en outre un buffer glissant, un mécanisme de versioning des modèles, un bootstrap empêchant les prédictions prématurées, ainsi qu'un réentraînement automatique à intervalles réguliers. L'apport principal de cette approche est de permettre à chaque agence d'apprendre à partir de ses propres données, tout en gardant la possibilité de partager les connaissances utiles entre les différents modèles.

Dans un second temps, FlowGuard intègre un mécanisme d'apprentissage fédéré basé sur une agrégation de type FedAvg. Cette stratégie consiste à combiner les paramètres des modèles locaux en fonction du nombre d'échantillons observés par chaque agence, afin de produire un modèle global plus robuste, sans centraliser les données brutes. Ce choix est important, car il répond à une contrainte opérationnelle majeure : les agences peuvent conserver leurs données localement tout en bénéficiant d'une amélioration collective du modèle. En parallèle, le système assure la traçabilité complète des transactions, des alertes, des modèles et des rounds fédérés dans Neo4j, ce qui facilite l'analyse historique, l'audit et la supervision. Enfin, un tableau de bord de monitoring en temps réel permet de visualiser les indicateurs clés, les alertes, l'évolution des modèles locaux et les rounds d'apprentissage fédéré. Ainsi, la solution proposée ne se limite pas à la détection de fraude ; elle fournit aussi une infrastructure d'observation et de pilotage du système.

L'apport global de ce projet réside donc dans la conception d'un système intégré, modulaire et évolutif, capable d'assurer à la fois le traitement temps réel, l'apprentissage local, l'agrégation fédérée et la supervision des événements. En combinant plusieurs technologies complémentaires, FlowGuard illustre une approche moderne de la détection de fraude, adaptée aux contraintes des environnements distribués et aux exigences de réactivité des services financiers.

Le reste de ce manuscrit est organisé de manière à présenter progressivement l'ensemble de la démarche suivie. Le chapitre suivant introduira le contexte général du projet, la problématique étudiée ainsi que le cahier des charges associé. Les chapitres ultérieurs détailleront l'architecture proposée, les choix techniques retenus, l'implémentation des différents modules, puis les tests et les résultats obtenus. Enfin, une conclusion générale viendra rappeler les apports du projet et ouvrir sur les perspectives d'amélioration possibles.

Chapitre 2 : Contexte et analyse

Introduction du chapitre

Ce chapitre présente le cadre dans lequel s’inscrit le projet **FlowGuard**, ainsi que les besoins auxquels il répond. Le système a été conçu pour les **agences de transfert d’argent**, dans un environnement où les transactions sont nombreuses, rapides et soumises à des risques de fraude qu’il faut détecter en temps réel. L’objectif n’est pas seulement de surveiller les opérations, mais aussi de construire une chaîne de traitement complète capable de recevoir des flux continus, de les nettoyer, de les analyser par apprentissage automatique, puis de conserver une trace exploitable des événements et des décisions prises. L’architecture globale du projet montre clairement cette logique de traitement distribué et continu, avec Kafka, des modules de nettoyage, des modèles locaux, un agrégateur fédéré et un stockage Neo4j, auxquels s’ajoute un tableau de bord de supervision.

2.2. Présentation du projet et contexte

Le projet FlowGuard s’inscrit dans le contexte de la **détection de fraude transactionnelle en temps réel** pour plusieurs agences de transfert d’argent. Il repose sur une simulation de trois agences distinctes, notées **A, B et C**, qui n’ont pas le même comportement transactionnel : l’agence A est stable, l’agence B est plus bruitée, et l’agence C est plus orientée vers le numérique. Cette différenciation est importante, car elle reflète un principe fondamental du domaine : une transaction normale dans une agence peut paraître inhabituelle dans une autre. Le système doit donc apprendre des comportements locaux tout en restant capable de partager de l’information utile entre les agences.

D’un point de vue technique, FlowGuard a été structuré comme un **pipeline temps réel**. Les transactions sont d’abord produites sous forme de flux Kafka dans un topic brut, puis nettoyées et validées avant d’être traitées par le module de machine learning. Le pipeline exploite ensuite un modèle local par agence, avec extraction de caractéristiques, prédiction, génération d’alertes et écriture dans Neo4j. En parallèle, un mécanisme d’**apprentissage fédéré** agrège les modèles locaux dès que les trois

agences disposent d'une nouvelle version non encore agrégée. Cette architecture permet de concilier traitement distribué, adaptation locale et mutualisation des connaissances, ce qui est particulièrement pertinent pour un environnement de transfert d'argent où la fraude évolue rapidement.

Le projet comprend également un **tableau de bord de supervision temps réel** construit avec Dash et connecté à Neo4j. Ce dashboard permet d'observer les indicateurs clés, les alertes, l'état des modèles locaux, les rounds fédérés et l'analyse des transactions. Dans un contexte métier, cet élément est essentiel, car une solution de détection de fraude ne doit pas seulement produire des scores, elle doit aussi rendre les résultats lisibles, interprétables et exploitables par un opérateur ou un responsable de supervision. L'ensemble du système constitue donc une chaîne complète, allant de la génération des données jusqu'à leur visualisation.

2.3. Cahier des charges

Le cahier des charges du projet peut être formulé autour de plusieurs exigences fonctionnelles et techniques.

2.3.1. Traitement des flux en temps réel

Le système doit être capable de recevoir des transactions en continu, de les transporter via Kafka, puis de les traiter sans rupture dans un flux opérationnel. Le projet prévoit ainsi cinq topics Kafka : transactions.raw, transactions.cleaned, alerts, model.updates et federated.updates. Cette organisation sépare les étapes du pipeline et permet de mieux contrôler les échanges entre les modules.

2.3.2. Nettoyage et validation des données

Avant toute exploitation analytique, les transactions doivent être contrôlées. Le module de nettoyage a pour rôle de valider les champs obligatoires, de normaliser les formats et de vérifier certaines contraintes métier, comme la cohérence des montants ou des valeurs temporelles. Cette étape est indispensable pour éviter que des données incohérentes perturbent les modèles de détection.

2.3.3. Extraction de caractéristiques

Le système doit transformer chaque transaction en un vecteur numérique de dimension fixe afin de l'exploiter par un algorithme de classification. Le projet retient **20 caractéristiques**, incluant des variables normalisées sur le montant, l'heure et le jour, des indicateurs booléens comme `is_foreign` et `is_online`, ainsi que des encodages one-hot pour les catégories marchandes et les localisations. Cette représentation donne au modèle une base exploitable et homogène.

2.3.4. Détection locale de fraude

Chaque agence doit disposer de son propre modèle local de détection. Le cahier des charges prévoit l'utilisation d'un `SGDClassifier` avec une perte logistique, un buffer glissant de taille limitée et un système de versioning automatique. Le modèle ne doit pas prédire trop tôt : un mécanisme de bootstrap impose un seuil minimal de transactions avant les premières prédictions. Le réentraînement est ensuite déclenché périodiquement, tous les 1000 événements, afin de suivre l'évolution des comportements.

2.3.5. Apprentissage fédéré

Le système doit aller au-delà du simple apprentissage local et permettre une consolidation globale des modèles. Pour cela, FlowGuard applique une logique de **FedAvg**, où les coefficients des modèles locaux sont agrégés de manière pondérée par le nombre d'échantillons observés. Le round fédéré ne se déclenche que lorsque les trois agences ont fourni une nouvelle version non encore agrégée. Cette exigence garantit une fusion cohérente des connaissances sans centraliser les données brutes.

2.3.6. Stockage et traçabilité

Toutes les transactions, alertes, modèles et rounds fédérés doivent être persistés dans une base orientée graphe. Neo4j est utilisé pour représenter les entités principales et leurs relations, notamment entre transaction, agence, modèle, alerte et round fédéré. Le schéma prévoit des contraintes d'unicité, des index et des opérations de type `MERGE` afin d'éviter les doublons et de maintenir une bonne qualité de stockage. Cette traçabilité est essentielle pour l'audit, le suivi historique et l'analyse postérieure des incidents.

2.3.7. Supervision visuelle

Le projet doit fournir une interface graphique permettant de suivre l'activité du système en temps réel. Le dashboard développé dans le dossier `flowguard_dashboard` répond à ce besoin avec plusieurs pages dédiées aux indicateurs généraux, aux alertes, aux modèles locaux, à l'apprentissage fédéré et à l'analyse des transactions. Le cahier des charges inclut donc une exigence de lisibilité et de supervision opérationnelle.

2.3.8. Modularité et maintenabilité

Le système doit être organisé de manière claire et maintenable. La séparation en deux ensembles — System pour le pipeline backend et `flowguard_dashboard` pour l'interface de monitoring — répond à cet objectif. À l'intérieur du backend, les responsabilités sont elles-mêmes séparées entre configuration, streams, ingestion, machine learning, fédération, stockage et export du schéma. Cette organisation facilite les tests, la mise à jour et l'extension du projet.

Conclusion du chapitre

Ce chapitre a permis de situer FlowGuard dans son contexte métier et technique. Le projet répond à une problématique concrète pour les **agences de transfert d'argent** : détecter des comportements frauduleux dans un flux continu de transactions, tout en tenant compte des différences entre agences et des contraintes d'un environnement distribué. Pour satisfaire ce besoin, la solution propose une architecture complète combinant Kafka, nettoyage des données, feature engineering, modèles locaux, apprentissage fédéré, stockage dans Neo4j et tableau de bord de supervision. L'analyse du contexte et du cahier des charges montre ainsi que le projet n'est pas seulement un exercice technique, mais une réponse structurée à une problématique de surveillance financière en temps réel.

Chapitre 3 : Modélisation

Introduction du chapitre

Ce chapitre présente la modélisation du système FlowGuard à travers plusieurs diagrammes UML permettant de représenter les différents aspects fonctionnels et techniques de l'architecture proposée. L'objectif de cette phase est de fournir une vision claire du fonctionnement global du système, des interactions entre ses composants et du déploiement des différents modules dans un environnement distribué.

La modélisation réalisée met en évidence le cycle complet de traitement des transactions financières, depuis leur génération par les streams jusqu'à leur visualisation dans le dashboard temps réel. Elle décrit également le fonctionnement du pipeline de Machine Learning, le mécanisme de Federated Learning utilisé pour l'agrégation des modèles locaux, ainsi que l'architecture de déploiement basée sur Kafka, Neo4j et Dash/Plotly.

Les diagrammes présentés dans ce chapitre permettent donc de mieux comprendre les échanges entre les composants, les flux de données temps réel, les mécanismes d'apprentissage et l'organisation distribuée du système FlowGuard.

3.2. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les principales fonctionnalités offertes par le système FlowGuard ainsi que les interactions entre les acteurs et les différents modules de traitement. Il met en évidence le cycle global de détection de fraude, depuis la génération des transactions jusqu'à leur analyse et leur visualisation dans le dashboard temps réel.

Le premier acteur représente les agences de transfert, qui génèrent des transactions simulées puis les publient dans Kafka afin d'alimenter le pipeline temps réel. Les transactions passent ensuite par une phase de nettoyage et de validation avant d'être

utilisées par le pipeline de Machine Learning pour produire des scores de fraude. Le système peut alors détecter les transactions suspectes, générer des alertes et réentraîner périodiquement les modèles locaux.

Le diagramme montre également le mécanisme de Federated Learning basé sur FedAvg, où les modèles locaux des différentes agences sont agrégés afin de produire un modèle global partagé. Les modèles locaux et globaux sont ensuite versionnés et sauvegardés pour assurer le suivi de l'apprentissage et la conservation des différentes versions des artefacts.

Enfin, l'analyste fraude interagit avec le dashboard afin de consulter les modèles, visualiser les indicateurs de performance, suivre les alertes en temps réel et analyser les transactions stockées dans Neo4j. Cette base graphe joue un rôle central dans le système, puisqu'elle conserve les transactions, les alertes, les modèles et les rounds de Federated Learning afin d'alimenter les fonctionnalités d'analyse et de supervision.

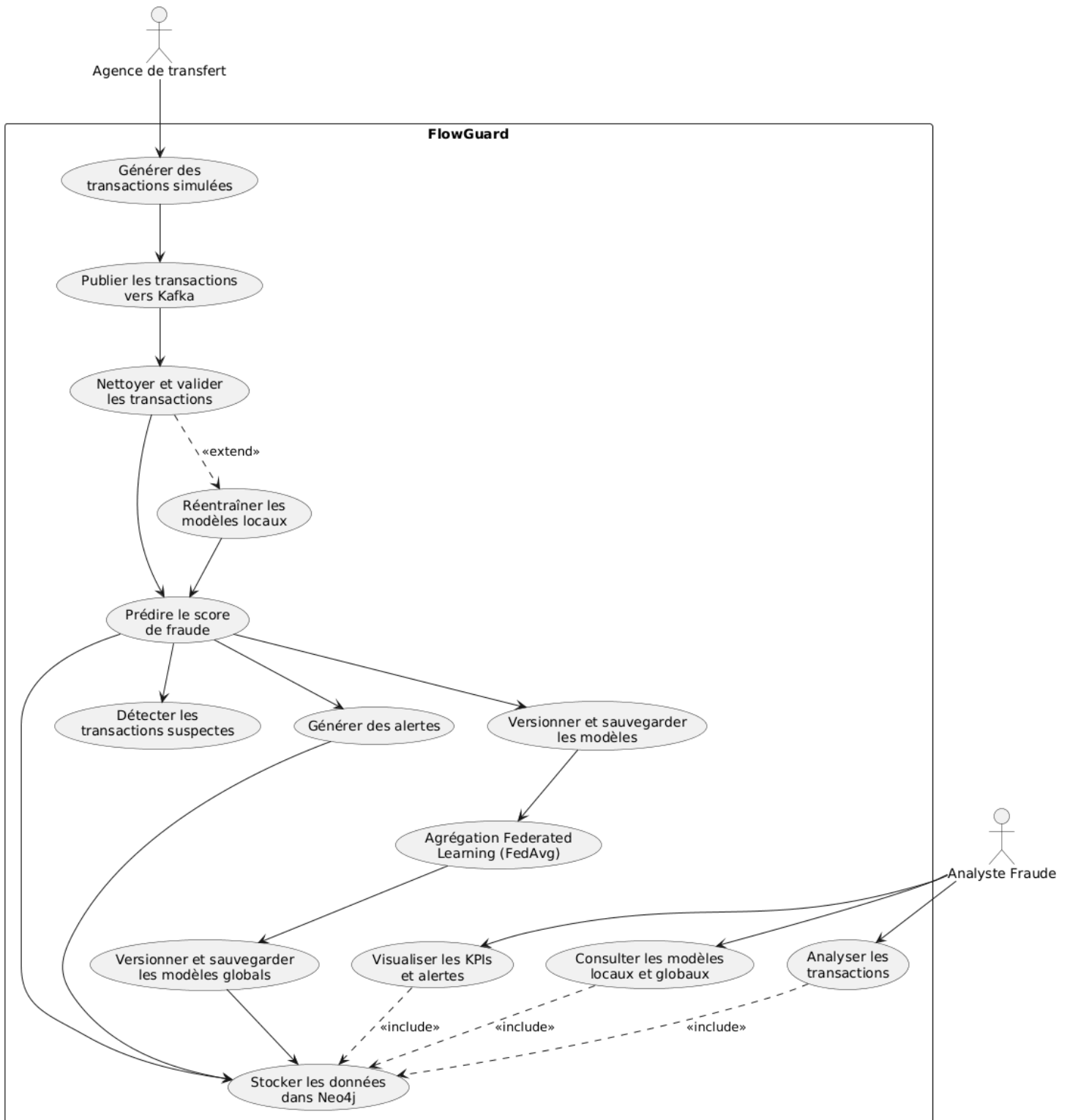


Figure 1 : Diagramme de cas d'utilisation

3.3. Diagramme de séquence : Streams et nettoyage

Ce diagramme de séquence illustre le fonctionnement de la première étape du pipeline temps réel de FlowGuard, correspondant à la génération des transactions et au nettoyage des données avant leur traitement par le pipeline de Machine Learning.

Le processus débute par les trois modules de génération FakeStream A, FakeStream B et FakeStream C, qui simulent en continu des transactions financières représentant les comportements spécifiques des différentes agences. Chaque stream publie ses transactions dans le topic Kafka transactions.raw, qui joue le rôle de couche de communication temps réel entre les différents composants du système.

Le module TransactionCleaner, appartenant au consumer group cleaner_group, consomme ensuite les transactions brutes depuis Kafka. Son rôle est de garantir la qualité et la cohérence des données avant leur utilisation par les modèles de Machine Learning. Pour cela, plusieurs opérations sont effectuées : vérification des champs obligatoires, validation des contraintes métier et normalisation des types et formats.

Une fois les contrôles terminés, deux cas sont possibles. Si la transaction est valide, elle est publiée dans le topic transactions.cleaned afin d'être utilisée par le pipeline ML. Dans le cas contraire, la transaction est rejetée afin d'éviter l'introduction de données incohérentes dans le système.

Ce mécanisme permet d'assurer un flux de données propre, standardisé et compatible avec les traitements temps réel réalisés dans les étapes suivantes du pipeline FlowGuard.

FlowGuard - Diagramme de séquence : Streams et nettoyage

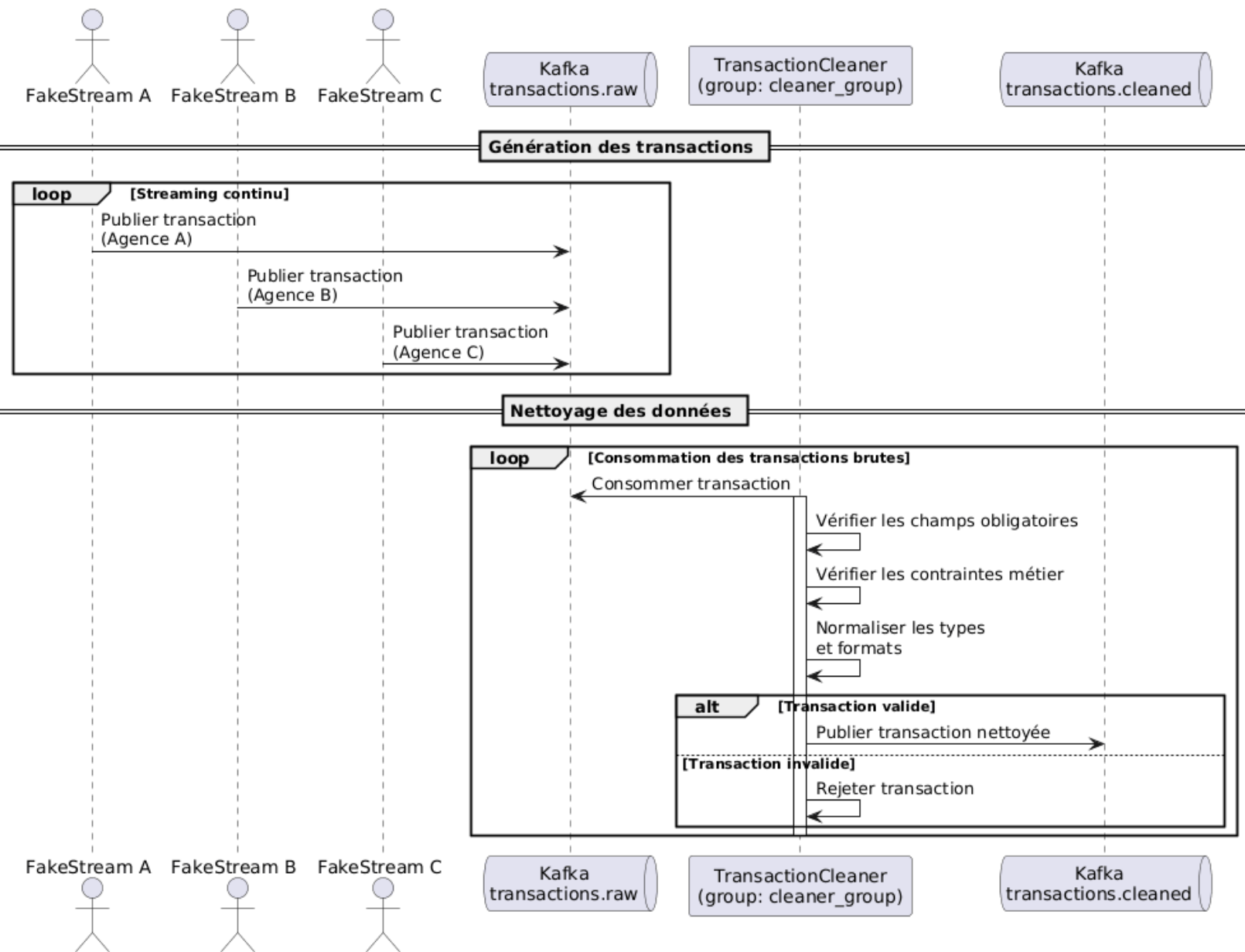


Figure 2 : Diagramme de séquence Streams et nettoyage

3.4. Diagramme de séquence : Pipeline ML

Ce diagramme de séquence présente le fonctionnement détaillé du pipeline de Machine Learning de FlowGuard, responsable de la détection de fraude en temps réel, du réentraînement des modèles locaux et du versionnement des artefacts.

La première étape correspond à l'initialisation des modèles locaux des agences A, B et C. Au démarrage du système, le pipeline vérifie l'existence d'une version sauvegardée des modèles dans le répertoire des artefacts. Si une version précédente existe, le dernier modèle ainsi que ses métadonnées sont chargés afin de reprendre l'apprentissage à partir de l'état le plus récent. Dans le cas contraire, un modèle vierge est créé et le système attend l'accumulation d'un minimum de 1000 transactions avant le premier entraînement.

Le diagramme montre ensuite le mécanisme d'entraînement initial puis de réentraînement périodique. Tous les 1000 événements par agence, le pipeline lance une nouvelle phase d'apprentissage à partir des données présentes dans le buffer local du modèle. Une fois l'entraînement terminé, le système calcule les métriques du modèle, génère une nouvelle version, sauvegarde les fichiers `model_vN.pkl` et `meta_vN.json`, puis publie une mise à jour dans le topic Kafka `model.updates`. Les informations du modèle sont également enregistrées dans Neo4j afin de conserver l'historique des versions et des performances.

La dernière partie du diagramme décrit la prédiction en temps réel. Le pipeline consomme les transactions nettoyées depuis le topic `transactions.cleaned`, identifie l'agence concernée puis applique le module de feature engineering afin de transformer la transaction en un vecteur numérique de 20 caractéristiques. La transaction est ensuite ajoutée au buffer du modèle local correspondant et un score de fraude est calculé grâce au modèle ML.

Les résultats de la prédiction sont enregistrés dans Neo4j sous forme de transactions et de prédictions. Si le score dépasse le seuil défini par le système, une alerte est publiée dans le topic Kafka `alerts` et une entité d'alerte est également stockée dans Neo4j.

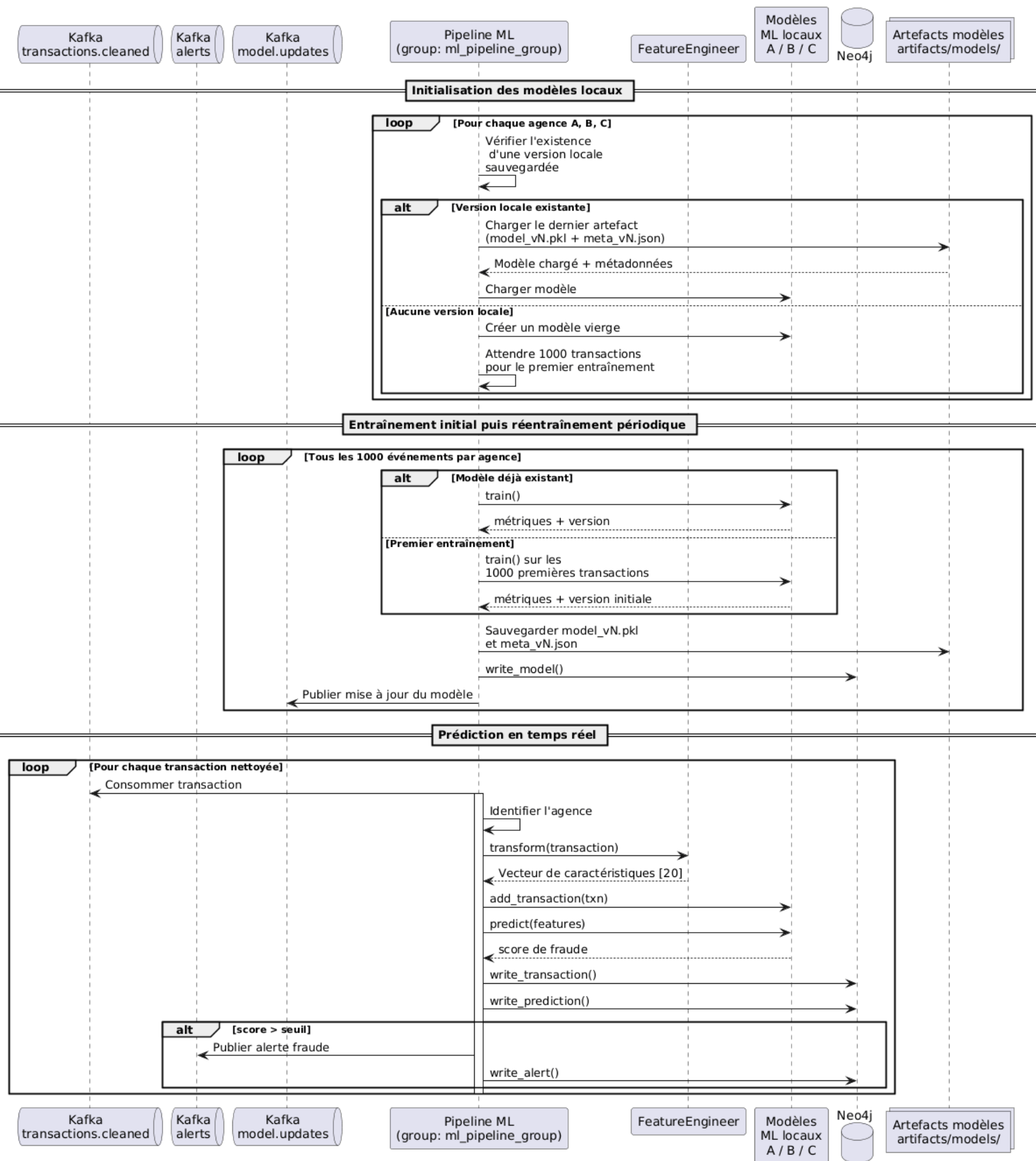


Figure 3 : Diagramme de séquence Pipeline ML

3.4. Diagramme de séquence : Federated Learning

Ce diagramme présente le fonctionnement du module de Federated Learning de FlowGuard, chargé d'agréger les modèles locaux des agences A, B et C afin de construire un modèle global partagé.

L'agrégateur fédéré consomme les mises à jour publiées dans le topic Kafka `model.updates` et met à jour les versions locales disponibles. Lorsque les trois agences possèdent chacune une nouvelle version de modèle, le processus d'agrégation est déclenché.

L'agrégateur récupère alors les poids des modèles locaux (`coef_`, `intercept_` et `n_samples`) puis les transmet au moteur FedAvg. Celui-ci applique une moyenne pondérée afin de construire un modèle global à partir des contributions des différentes agences.

Une fois l'agrégation terminée, le modèle global est sauvegardé dans le répertoire des artefacts globaux puis une mise à jour est publiée dans le topic `federated.updates`. Les informations du round fédéré sont également enregistrées dans Neo4j afin de conserver l'historique des versions et des métriques.

Ce mécanisme permet à plusieurs agences de collaborer à l'apprentissage d'un modèle commun sans partager directement leurs données sensibles.

FlowGuard - Diagramme de séquence : Federated Learning

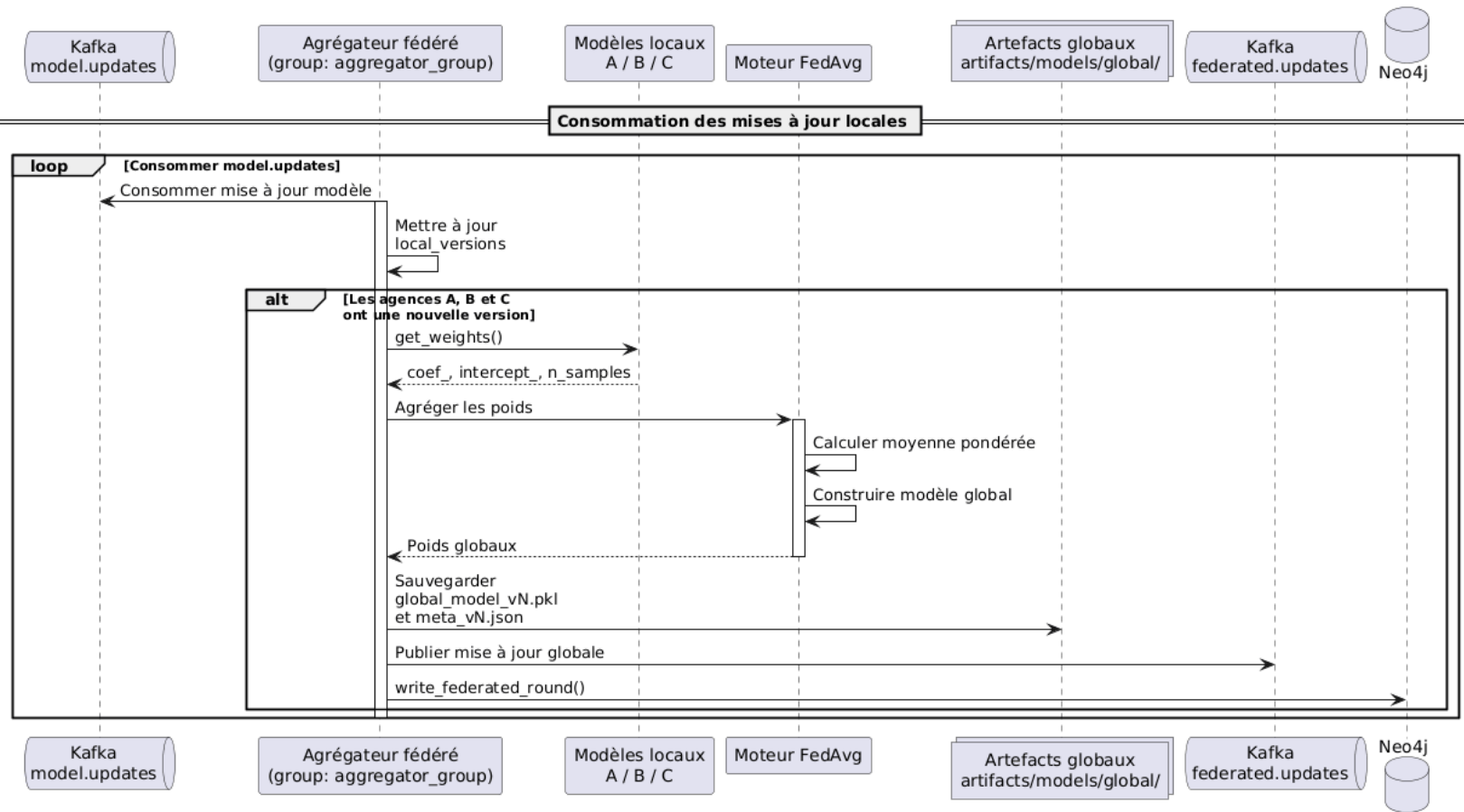


Figure 4 : Diagramme de séquence Federated Learning

3.4. Diagramme de séquence : Dashboard temps réel

Ce diagramme présente le fonctionnement du dashboard temps réel de FlowGuard, utilisé pour visualiser les transactions, les alertes, les métriques des modèles et les résultats du Federated Learning.

Lorsqu'un utilisateur ouvre le dashboard, l'application Dash charge la page sélectionnée puis demande les données nécessaires au module Neo4jDataLoader. Celui-ci vérifie d'abord le cache TTL afin d'éviter des requêtes inutiles vers Neo4j. Si les données sont encore présentes en mémoire, elles sont directement retournées ; sinon, le loader exécute les requêtes Cypher sur Neo4j puis stocke les résultats dans le cache avant de les retourner au dashboard.

Les données récupérées sont ensuite utilisées pour construire les graphiques, les KPIs et les tableaux affichés dans l'interface utilisateur.

Le diagramme montre également le mécanisme de rafraîchissement automatique du dashboard. Toutes les 10 secondes, les pages rechargent les métriques et vérifient l'expiration du cache. Si le cache a expiré, de nouvelles requêtes sont exécutées sur Neo4j ; sinon, les données déjà mises en mémoire sont réutilisées. Les composants du dashboard sont ensuite mis à jour afin d'afficher les informations les plus récentes en temps réel.

FlowGuard - Diagramme de séquence : Dashboard temps réel

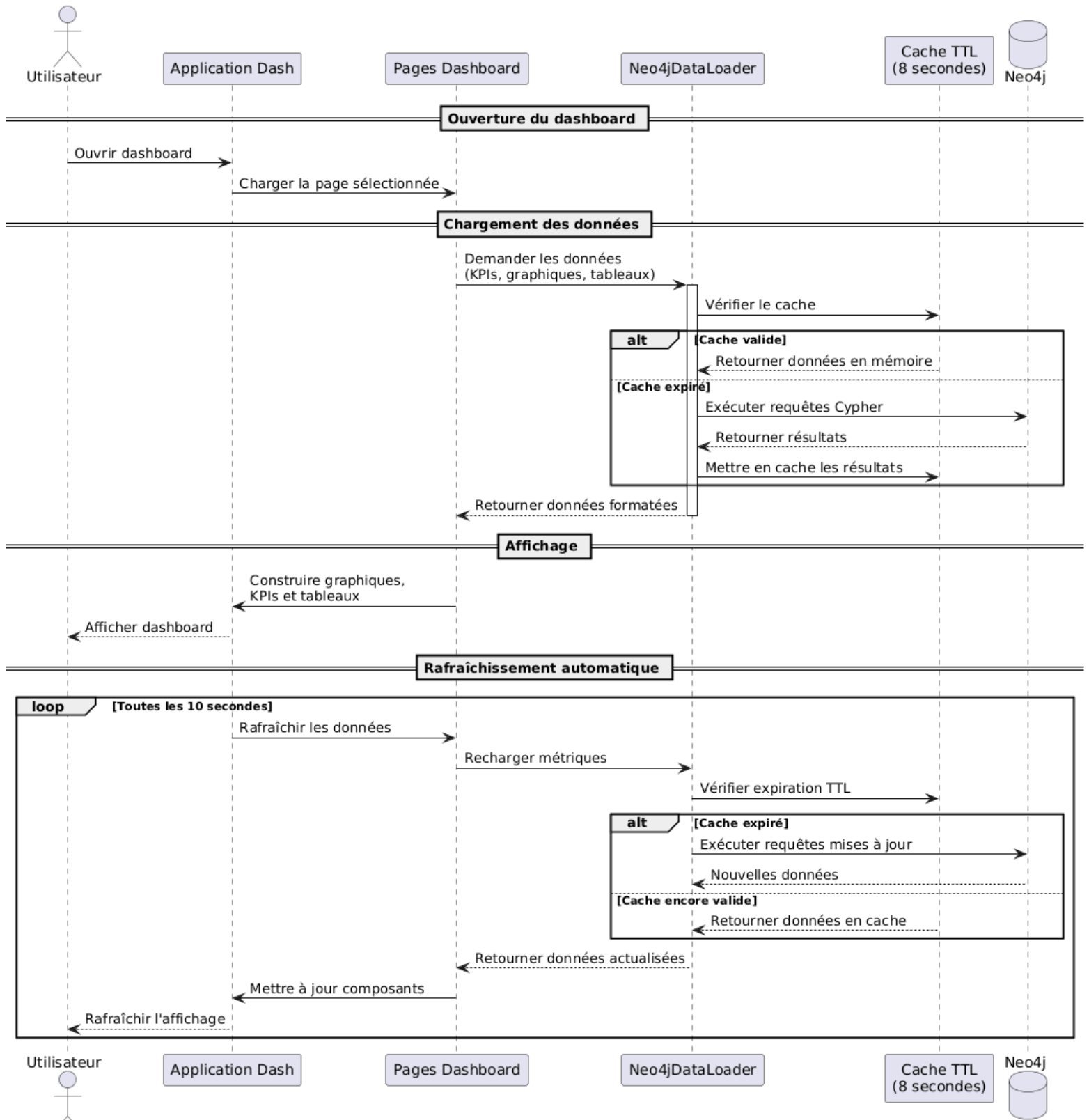


Figure 5 : Diagramme de séquence Dashboard temps réel

3.4. Diagramme de déploiement

Ce diagramme présente l'architecture de déploiement du système FlowGuard ainsi que la répartition des composants entre les serveurs locaux des agences et le serveur central.

Chaque agence dispose d'un serveur local contenant un module de génération de transactions (Stream), un modèle ML local et un espace de stockage des artefacts des modèles. Les transactions générées sont publiées dans Kafka via le topic `transactions.raw`.

Le serveur central FlowGuard contient les principaux composants du système : le Transaction Cleaner, le Pipeline ML et le module Federated Learning Aggregator. Ces composants assurent le nettoyage des données, les prédictions de fraude, la génération des alertes et l'agrégation des modèles locaux.

Apache Kafka permet la communication temps réel entre les différents modules à travers plusieurs topics, tandis que Neo4j stocke les transactions, alertes, modèles et rounds de Federated Learning. Enfin, le dashboard Dash/Plotly permet de visualiser les données et les indicateurs du système en temps réel.

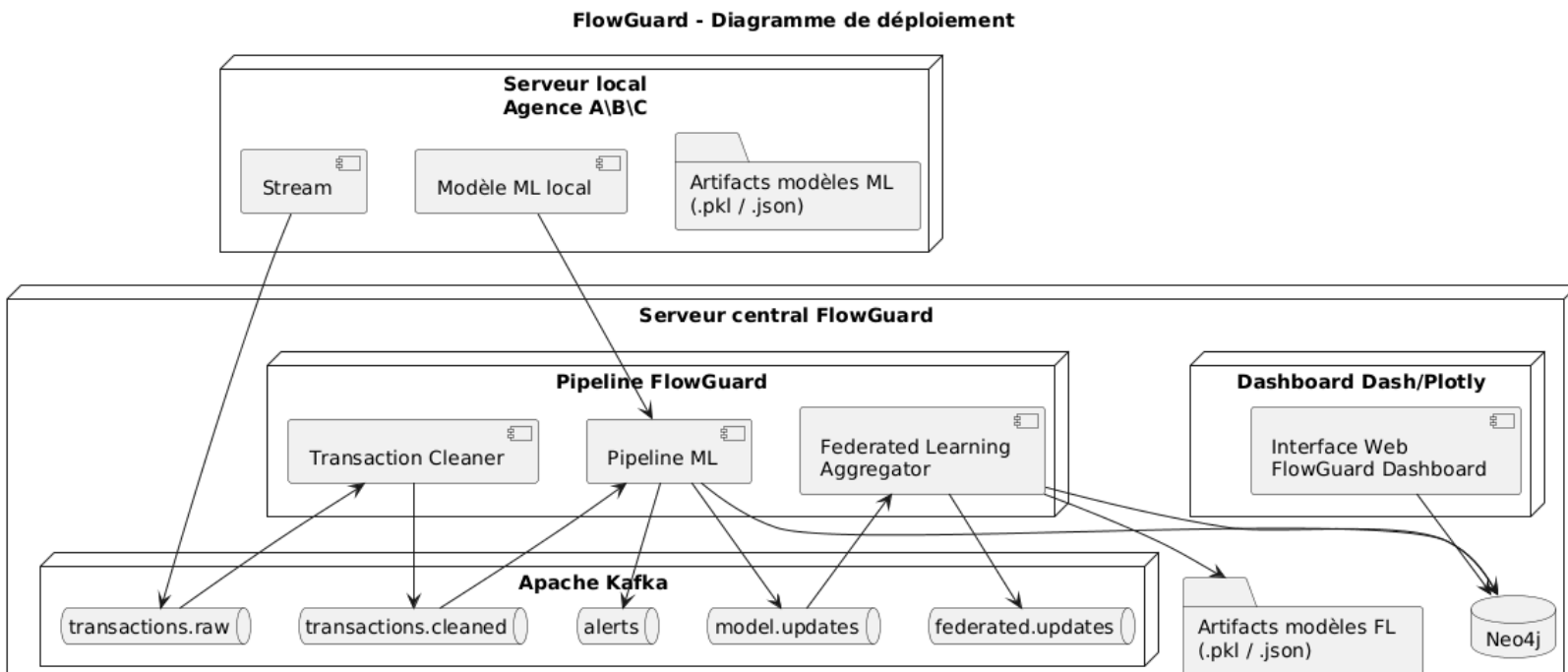


Figure 6 : Diagramme de déploiement

Conclusion du chapitre

Dans ce chapitre, nous avons présenté la modélisation UML du système FlowGuard à travers différents diagrammes représentant les interactions, les flux de données et l'architecture de déploiement du projet. Cette modélisation a permis de visualiser le fonctionnement global du pipeline temps réel, depuis la génération des transactions jusqu'à l'affichage des résultats dans le dashboard.

Les diagrammes de séquence ont mis en évidence les différentes étapes de traitement des données, notamment le nettoyage des transactions, le pipeline de Machine Learning local, le mécanisme de Federated Learning et le fonctionnement du dashboard de supervision. Le diagramme de déploiement a également permis de représenter l'organisation distribuée des composants entre les serveurs locaux des agences et le serveur central.

Cette phase de modélisation constitue une base importante pour l'implémentation et l'analyse technique du système, tout en facilitant la compréhension des échanges entre les différents modules de FlowGuard.

Chapitre 4 : Outils technologiques et réalisation

Introduction du chapitre

Ce chapitre présente les outils technologiques utilisés pour la réalisation du projet FlowGuard ainsi que les différentes étapes de son implémentation. Le choix des technologies a été effectué en fonction des besoins du système, notamment le traitement temps réel des données, l'apprentissage automatique, le Federated Learning, le stockage des informations et la visualisation des résultats.

La première partie du chapitre décrit les principaux outils et frameworks utilisés durant le développement du projet. La seconde partie présente la réalisation pratique du système, depuis la génération des transactions jusqu'à l'affichage des données dans le dashboard temps réel.

4.2. Outils technologiques

4.2.1. Outils de développement

4.2.1.1. Python



Figure 7 : Python logo

Python constitue le langage principal utilisé pour le développement du projet FlowGuard. Il a été choisi grâce à sa simplicité, sa flexibilité ainsi que la richesse de son écosystème dédié au Big Data, au Machine Learning et au développement d'applications temps réel. L'ensemble des modules du système, notamment les

streams, le pipeline ML, le Federated Learning et le dashboard, ont été développés en Python.

Référence :

<https://www.python.org/>

4.2.1.2. Apache Kafka



Figure 8 : Kafka logo

Apache Kafka est utilisé comme plateforme de streaming et de communication temps réel entre les différents composants du système. Il permet la publication et la consommation des flux de transactions via plusieurs topics tels que transactions.raw, transactions.cleaned, alerts, model.updates et federated.updates.

Kafka joue un rôle central dans l'architecture FlowGuard en assurant un échange rapide, distribué et scalable des données entre les modules.

Référence :

<https://kafka.apache.org/>

4.2.1.3. Scikit-learn



Figure 9 : Scikit-learn logo

Scikit-learn est la bibliothèque de Machine Learning utilisée pour l'entraînement des modèles locaux de détection de fraude. Le projet utilise principalement SGDClassifier avec log_loss afin de produire des scores probabilistes de fraude adaptés au traitement temps réel.

Cette bibliothèque a été choisie pour sa simplicité d'utilisation, ses performances et sa compatibilité avec le Federated Learning.

Référence :

<https://scikit-learn.org/>

4.2.1.4. Neo4j



Figure 10 : Neo4j logo

Neo4j est une base de données graphe utilisée pour stocker les transactions, alertes, modèles ML et rounds de Federated Learning. Elle permet de représenter les relations entre les différentes entités du système et facilite l'analyse des données en temps réel.

Neo4j alimente également le dashboard de supervision grâce aux requêtes Cypher exécutées par l'application.

Référence :

<https://neo4j.com/>

4.2.1.5. Dash et Plotly



Figure 11 : Dash - Plotly logo

Dash est un framework Python utilisé pour le développement du dashboard web du projet. Associé à Plotly, il permet de créer des interfaces interactives contenant des graphiques, tableaux et indicateurs temps réel.

Le dashboard FlowGuard permet notamment de visualiser les alertes, les transactions, les métriques des modèles locaux et les résultats du Federated Learning.

Références

:

<https://dash.plotly.com/>

<https://plotly.com/>

4.2.1.6. NumPy



Figure 12 : NumPy logo

NumPy est utilisé pour la manipulation des tableaux numériques et des vecteurs de caractéristiques générés lors du feature engineering. Cette bibliothèque facilite les opérations mathématiques nécessaires à l'entraînement des modèles de Machine Learning et à l'agrégation des poids dans le Federated Learning.

Référence

:

<https://numpy.org/>

4.2.2. Outils de conception

4.2.2.1. uml



Figure 13 : Logo UML

UML (Unified Modeling Language) a été utilisé pour modéliser les différents aspects structurels et fonctionnels du projet : diagramme de cas d'utilisation, diagramme de classes, diagrammes de séquence.

Ces modèles ont permis de clarifier les interactions entre les modules, les objets métier et les fonctionnalités de l'interface.

4.2.2.2. Plantuml



Figure 14 : Logo PlantUML

PlantUML a été utilisé pour générer les diagrammes UML de manière textuelle. Cet outil permet : une création rapide et modifiable de diagrammes, une intégration facile dans les rapports et une version textuelle du diagramme compatible avec les outils de versionnement.

4.3. Réalisation

4.3.1. Architecture générale du système

L'architecture de FlowGuard repose sur une approche distribuée et temps réel combinant Apache Kafka, Machine Learning, Federated Learning, Neo4j et un dashboard interactif. Les différents modules du système communiquent entre eux via des flux Kafka afin d'assurer un traitement continu des transactions financières.

Le fonctionnement général du système suit les étapes suivantes :

- Les streams des agences A, B et C génèrent des transactions simulées puis les publient dans le topic `transactions.raw`.
- Le module `TransactionCleaner` consomme les données brutes, effectue le nettoyage et la validation des transactions, puis republie les données propres dans `transactions.cleaned`.
- Le pipeline de Machine Learning réalise le `feature engineering` et calcule un score de fraude pour chaque transaction à l'aide des modèles locaux.
- Si une transaction dépasse le seuil de fraude défini, une alerte est générée et publiée dans le topic `alerts`.
- Les modèles locaux sont réentraînés périodiquement, puis leurs mises à jour sont publiées dans `model.updates`.
- Le module de Federated Learning applique l'algorithme `FedAvg` afin de construire un modèle global à partir des modèles locaux des agences.
- Toutes les transactions, alertes et informations des modèles sont stockées dans Neo4j afin d'alimenter le dashboard temps réel.

Enfin, un dashboard développé avec Dash et Plotly permet de superviser le système grâce à des graphiques, indicateurs et tableaux interactifs affichant les alertes, les transactions et les performances des modèles ML.

4.3.2. Mise en place de l'environnement de développement

La mise en place de l'environnement de développement de FlowGuard a nécessité l'installation et la configuration de plusieurs outils permettant d'assurer le

fonctionnement du pipeline temps réel, du Machine Learning et du dashboard de supervision.

L'environnement utilisé repose principalement sur Python ainsi que plusieurs bibliothèques spécialisées pour le streaming, le Machine Learning et la visualisation des données. Les dépendances du projet sont centralisées dans les fichiers requirements.txt afin de faciliter l'installation des packages nécessaires.

Les principales étapes de configuration sont les suivantes :

- Installation de Python et des bibliothèques du projet (kafka-python, scikit-learn, numpy, neo4j, dash, plotly, etc.).
- Installation et configuration d'Apache Kafka ainsi que Zookeeper pour assurer la communication temps réel entre les modules.
- Création des différents topics Kafka utilisés dans le système :
 - transactions.raw
 - transactions.cleaned
 - alerts
 - model.updates
 - federated.updates
- Installation et configuration de Neo4j afin de stocker les transactions, alertes, modèles et rounds de Federated Learning.
- Mise en place du dashboard Dash/Plotly pour la supervision et la visualisation des données temps réel.

Le projet est organisé en deux parties principales :

- System/ : contient les modules du pipeline temps réel, du Machine Learning et du Federated Learning ;
- flowguard_dashboard/ : contient l'application web de supervision et les composants du dashboard.

Cette organisation permet de séparer les traitements backend de l'interface de visualisation tout en facilitant la maintenance et l'évolution du système.

4.3.3. Configuration d'Apache Kafka et des topics

Apache Kafka est utilisé dans FlowGuard comme plateforme de communication temps réel entre les différents composants du système. Il permet d'assurer la transmission continue des transactions, alertes et mises à jour des modèles entre les producteurs et les consommateurs.

Après l'installation de Kafka et de Zookeeper, plusieurs topics ont été créés afin d'organiser les échanges de données dans le système :

- transactions.raw : contient les transactions brutes générées par les streams des agences ;
- transactions.cleaned : contient les transactions nettoyées et validées ;
- alerts : contient les alertes de fraude détectées ;
- model.updates : contient les mises à jour des modèles locaux ;
- federated.updates : contient les mises à jour des modèles globaux après agrégation fédérée.

L'architecture Kafka de FlowGuard repose sur plusieurs producteurs et consommateurs :

Producteurs :

- Les streams A, B et C publient les transactions dans transactions.raw.
- Le Transaction Cleaner publie les données nettoyées dans transactions.cleaned.
- Le pipeline ML publie les alertes dans alerts ainsi que les mises à jour des modèles dans model.updates.
- Le module Federated Learning publie les modèles globaux dans federated.updates.

Consommateurs :

- Le Transaction Cleaner consomme transactions.raw.
- Le pipeline ML consomme transactions.cleaned.
- Le module Federated Learning consomme model.updates.

- Le module Neo4j Writer consomme plusieurs topics afin de stocker les données dans Neo4j.

Chaque consommateur appartient également à un consumer group spécifique, comme `cleaner_group`, `ml_pipeline_group` et `aggregator_group`, afin d'organiser le traitement des flux et d'assurer une bonne répartition des tâches.

Cette configuration permet à FlowGuard de disposer d'une architecture temps réel scalable, distribuée et découplée, facilitant la communication entre les différents modules du système.

FlowGuard - Architecture Kafka

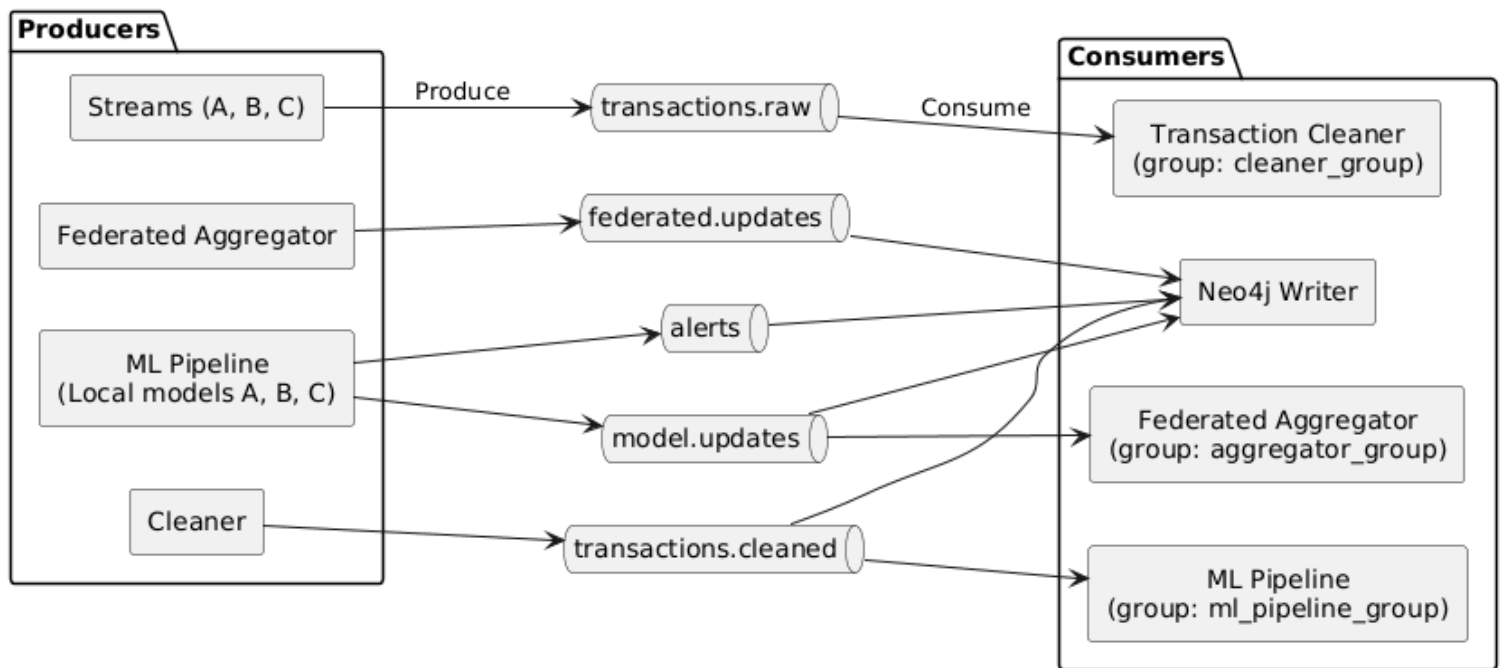


Figure 15 : Architecture Kafka

4.3.4. Génération et nettoyage des transactions simulées (Streams)

La première étape du système FlowGuard consiste à générer des transactions financières simulées afin d'alimenter le pipeline temps réel. Cette tâche est assurée par trois streams représentant différents profils d'agences de transfert. Les transactions générées sont publiées dans le topic Kafka `transactions.raw`.

Chaque stream possède un comportement spécifique :

- **Stream A** représente une agence stable avec des montants modérés et un moyen taux de fraude ;
- **Stream B** simule une agence plus risquée avec des montants élevés avec un haut taux de fraude ;
- **Stream C** représente une agence digitale avec des transactions fréquentes et principalement en ligne avec et un très faible taux de fraude.

Le système ne génère pas uniquement des données aléatoires. Plusieurs règles réalistes de fraude sont également intégrées :

- transactions nocturnes ;
- montants inhabituels ;
- opérations étrangères ;
- séries rapides de transactions (bursts).

Après leur génération, les transactions sont consommées par le module TransactionCleaner, chargé de nettoyer et valider les données avant leur utilisation par le pipeline ML. Ce module vérifie les champs obligatoires, les types de données et les contraintes métier afin d'assurer la cohérence des informations.

Validations métier :

- Le cleaner rejette les transactions si :
- le montant est inférieur ou égal à 0 ;
- l'heure est hors plage 0–23 ;
- le jour est hors plage 0–6 .

Le cleaner réalise également plusieurs opérations de normalisation :

- conversion des montants en float ;
- normalisation des catégories et localisations ;
- vérification des valeurs horaires et des jours de la semaine.

Les transactions valides sont ensuite publiées dans le topic transactions.cleaned, tandis que les données invalides sont rejetées afin de préserver la stabilité du

système. Cette étape garantit l'utilisation de données propres et cohérentes pour les traitements de Machine Learning.

4.3.5. Réalisation du pipeline de Machine Learning

Le pipeline de Machine Learning de FlowGuard assure la détection de fraude en temps réel à partir des transactions déjà nettoyées. Il consomme les messages du topic `transactions.cleaned`, transforme chaque transaction en un vecteur de **20 features**, puis calcule un score de fraude à l'aide d'un modèle supervisé binaire basé sur **SGDClassifier(loss='log_loss')**. Ce choix permet d'obtenir une sortie probabiliste adaptée à un traitement rapide et compatible avec le Federated Learning.

Le fonctionnement du pipeline suit une logique simple : la transaction est ajoutée au buffer local du modèle, transformée par le module de feature engineering, puis évaluée afin de produire une prédiction. Si le score dépasse le seuil défini, une alerte est générée et les résultats sont enregistrés dans Neo4j. Le pipeline réentraîne également les modèles locaux de manière périodique, tous les **1000 événements**, puis publie une mise à jour dans Kafka via le topic `model.updates`.

Chaque modèle local dispose de son propre historique, de son buffer glissant de taille maximale **5000**, et d'un versionnage automatique. Au démarrage, le système charge la dernière version disponible si elle existe ; sinon, il attend l'accumulation d'un premier lot de **1000 transactions** avant le premier entraînement. Une fois l'entraînement terminé, le système calcule plusieurs métriques telles que :

- accuracy ;
- precision ;
- recall ;
- f1-score ;
- taux de fraude.

Après chaque réentraînement, une nouvelle version du modèle est générée puis sauvegardée sous forme de fichiers `model_vN.pkl` et `meta_vN.json`. Les informations de version et les métriques sont également publiées dans Kafka via le topic `model.updates`.

Le feature engineering constitue une étape essentielle du pipeline. Chaque transaction est convertie en 20 dimensions : montant normalisé, heure normalisée, jour normalisé, deux variables booléennes (is_foreign, is_online), neuf variables pour les catégories marchandes et six pour les localisations. Cette représentation numérique garantit une entrée stable, sans valeurs NaN ou Inf, pour l'apprentissage et la prédiction.

Ce pipeline constitue ainsi le cœur du système de détection, puisqu'il combine nettoyage logique, apprentissage supervisé, scoring temps réel, génération d'alertes et réentraînement périodique des modèles locaux.

4.3.6. Gestion des modèles locaux et réentraînement

Le système FlowGuard utilise un modèle local indépendant pour chaque agence afin de conserver des comportements d'apprentissage adaptés aux profils transactionnels spécifiques de chaque environnement. Chaque modèle possède son propre espace de stockage, son historique de versions ainsi que ses métriques d'entraînement.

Le système met en place un mécanisme de versionnage automatique permettant de générer des versions successives des modèles (v1, v2, v3, etc.). Après chaque phase d'apprentissage, les artefacts du modèle sont sauvegardés dans des fichiers .pkl accompagnés de fichiers .json contenant les métadonnées et les métriques associées.

Afin de permettre une adaptation continue aux nouveaux comportements de fraude, les modèles sont réentraînés périodiquement à partir des transactions récentes conservées dans le buffer glissant. Cette approche permet d'éviter un apprentissage sur l'ensemble de l'historique tout en gardant des modèles capables d'évoluer avec le temps.

Le système prévoit également une phase de bootstrap au démarrage. Lorsqu'aucun modèle sauvegardé n'est disponible, le modèle attend l'accumulation d'un volume minimal de transactions avant le premier entraînement. Cette stratégie améliore la stabilité des prédictions et réduit les risques liés à un apprentissage sur des données insuffisantes.

Enfin, les modèles locaux exposent des fonctions d'échange de poids (`get_weights()` et `set_weights()`) permettant leur intégration avec le module de Federated Learning sans partager directement les données sensibles des agences.

4.3.7. Implémentation du Federated Learning avec FedAvg

Le module de Federated Learning de FlowGuard permet d'agréger les modèles locaux des agences sans centraliser leurs données brutes. Chaque agence entraîne son modèle en local, puis publie uniquement les mises à jour de ce modèle dans le topic Kafka `model.updates`.

L'agrégateur fédéré consomme ces mises à jour avec le consumer group `aggregator_group`. Il surveille l'état des versions locales et déclenche l'agrégation lorsque les trois agences A, B et C disposent chacune d'une nouvelle version non encore traitée.

L'agrégation repose sur l'algorithme **FedAvg**. Pour chaque modèle local, l'agrégateur récupère les coefficients `coef_`, l'intercept `intercept_` et le nombre d'échantillons `n_samples`. Ces paramètres sont ensuite combinés par moyenne pondérée, de sorte que les agences ayant davantage de données contribuent plus fortement au modèle global.

Une fois l'agrégation terminée, le modèle global est sauvegardé dans `artifacts/models/global/`, puis une notification est publiée dans `federated.updates`. Enfin, un nœud `FederatedRound` est enregistré dans Neo4j afin de conserver l'historique des rounds fédérés.

Cette approche permet d'améliorer la robustesse globale du système tout en respectant la confidentialité des données propres à chaque agence.

4.3.8. Intégration de Neo4j et stockage des données

Neo4j est utilisé dans FlowGuard comme base de données graphe afin de stocker les principales entités du système et surtout les relations entre elles. Cette approche est adaptée au contexte de détection de fraude, car elle permet de relier facilement

les transactions, les alertes, les modèles locaux, les agences et les rounds de Federated Learning.

L'écriture dans Neo4j est assurée par une classe dédiée, Neo4jWriter, qui initialise le schéma de la base, définit les contraintes d'unicité et crée les index nécessaires pour améliorer les performances des requêtes. Les opérations d'insertion utilisent principalement MERGE, ce qui permet d'éviter les doublons et de mettre à jour les nœuds existants si besoin.

Le système enregistre notamment :

- les **transactions** avec leurs caractéristiques et leur score de fraude ;
- les **alertes** générées lors de la détection d'un comportement suspect ;
- les **agences** ;
- les **modèles locaux** et leurs métriques ;
- les **rounds de Federated Learning**.

Les relations principales du graphe sont également conservées dans Neo4j, par exemple :

- une transaction appartient à une agence ;
- une transaction peut déclencher une alerte ;
- une transaction est évaluée par un modèle ;
- un modèle appartient à une agence ;
- un round fédéré agrège plusieurs modèles locaux.

Cette intégration permet de centraliser l'historique du système et d'alimenter efficacement le dashboard temps réel grâce aux requêtes Cypher.

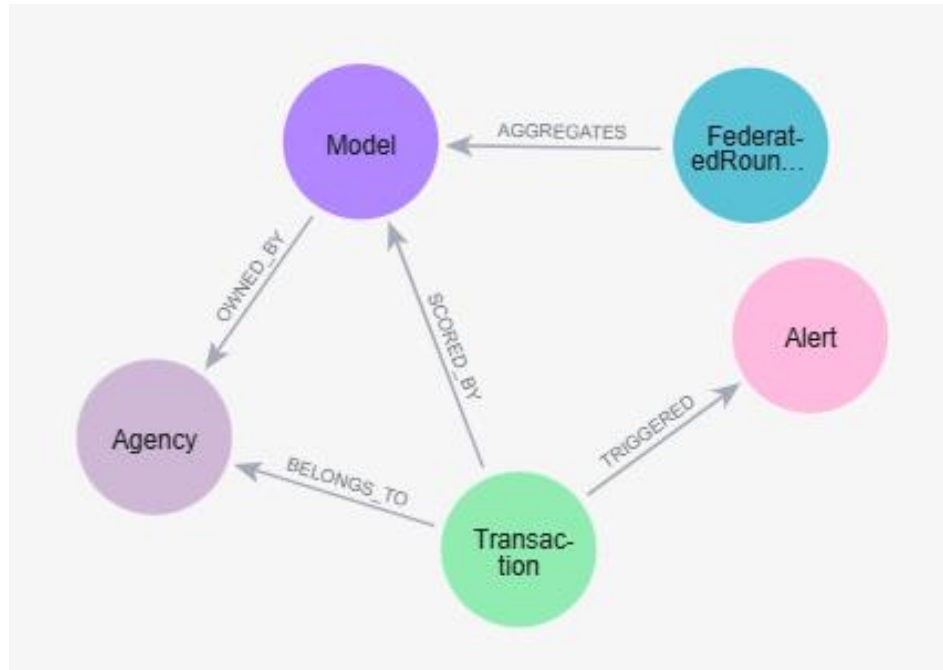


Figure 16 : Shema neo4j

4.3.9. Développement du dashboard temps réel

Le dashboard de FlowGuard a été développé avec **Dash/Plotly** afin d'offrir une interface de supervision en temps réel du système. Il se connecte à Neo4j pour récupérer les données stockées et les afficher sous forme de cartes KPI, graphiques interactifs, tableaux et indicateurs de statut.

L'application est structurée de manière modulaire :

- app.py sert de point d'entrée ;
- core/ contient la configuration, le cache TTL et le chargeur Neo4j ;
- components/ regroupe les éléments réutilisables de l'interface ;
- pages/ contient les différentes vues du dashboard.

Pour limiter les requêtes inutiles, le dashboard utilise un **cache TTL de 8 secondes**, ce qui améliore les performances tout en gardant une actualisation fréquente des données. En cas d'indisponibilité de Neo4j, l'interface affiche des états vides ou des messages d'attente afin de conserver une présentation claire.

L'interface comprend une barre latérale de navigation, un indicateur LIVE, un statut de connexion Neo4j et plusieurs pages dédiées :

- vue générale ;
- alertes temps réel ;
- modèles locaux ;
- Federated Learning ;
- analyse des transactions.

Cette réalisation permet de suivre en continu les transactions, les alertes, les performances des modèles et les rounds fédérés, tout en offrant une lecture synthétique et interactive du système FlowGuard.

4.3.10. Présentation des interfaces de l'application

Page Overview — Vue générale

La page **Overview** constitue l'interface principale du dashboard FlowGuard. Elle offre une vue synthétique et temps réel de l'état global du système de détection de fraude. Cette interface a été développée avec Dash/Plotly et utilise un thème sombre avec des composants de type glassmorphism afin d'améliorer la lisibilité des indicateurs.

L'interface est composée d'une barre latérale fixe contenant le logo FlowGuard, les liens de navigation, un badge LIVE ainsi qu'un indicateur de connexion Neo4j. La zone principale affiche les informations de supervision et se rafraîchit automatiquement toutes les 10 secondes.

La première partie de la page contient quatre cartes KPI affichant :

- le nombre total de transactions ;
- le nombre total d'alertes ;
- le taux global de fraude ;
- le nombre de rounds de Federated Learning.

Ces indicateurs permettent d'obtenir rapidement une vision globale de l'activité du système et de son état actuel.

La deuxième partie contient deux visualisations principales :

- un graphique en courbes représentant le flux de transactions par heure et par agence ;
- une jauge affichant le score moyen de fraude avec un seuil d'alerte fixé à 0.5.

Le graphique temporel permet d'observer les variations d'activité des agences A, B et C au cours de la journée, tandis que la jauge fournit une estimation globale du niveau de risque actuel.

Cette page joue ainsi le rôle de centre de supervision principal du système FlowGuard en regroupant les indicateurs essentiels de fonctionnement et de détection de fraude.

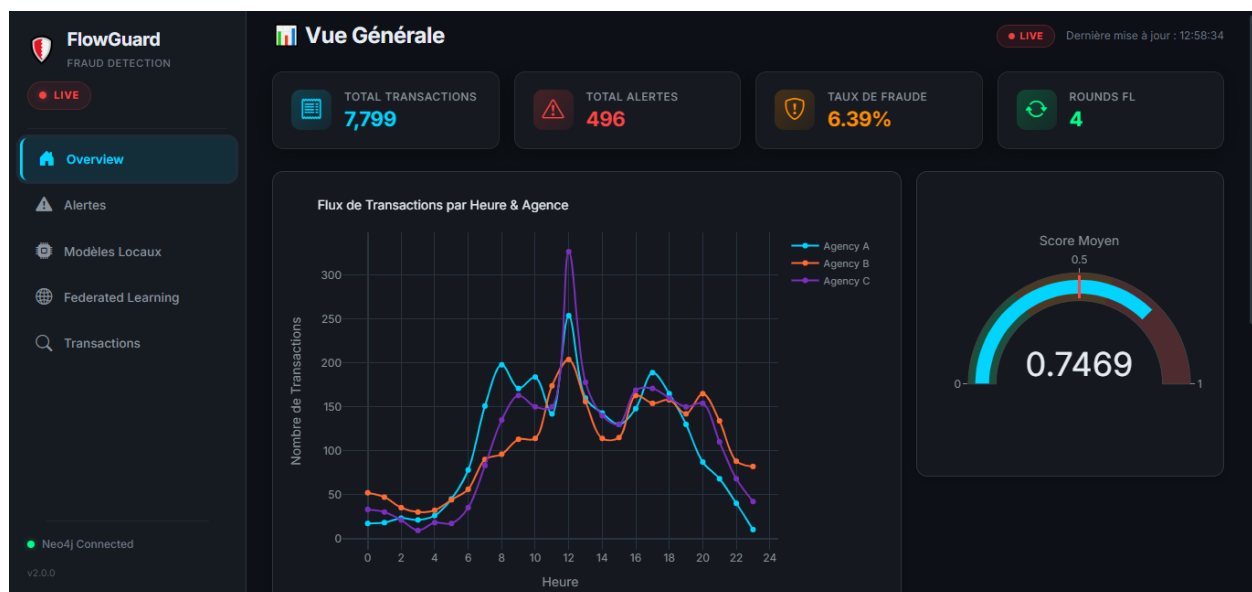


Figure 17 : Overview_1

La dernière partie de la page Overview contient deux visualisations complémentaires permettant d'analyser les alertes et les comportements des transactions.

Le premier graphique est un diagramme en barres représentant le nombre d'alertes générées par chaque agence. Chaque barre utilise une couleur spécifique à l'agence concernée (cyan pour

Agency A, orange pour Agency B et violet pour Agency C). Cette visualisation permet d'identifier rapidement les agences produisant le plus grand nombre de transactions suspectes.

Le second graphique est un nuage de points représentant la relation entre le montant des transactions et le score de fraude calculé par les modèles ML. L'axe horizontal correspond au montant de la transaction tandis que l'axe vertical représente le score de fraude compris entre 0 et 1. Les points sont colorés selon l'agence afin de distinguer les différents comportements transactionnels.

Cette visualisation permet d'observer la dispersion des transactions, d'identifier les montants associés aux scores élevés de fraude et de comparer les profils de risque entre les différentes agences.



Figure 18 : Overview_2

Page Alertes — Alertes Temps Réel

La page **Alertes Temps Réel** est dédiée au suivi et à l'analyse des alertes de fraude générées par le pipeline ML de FlowGuard. Elle permet de surveiller en temps réel les transactions suspectes détectées par les modèles locaux.

La première partie de l'interface contient trois cartes KPI affichant :

- le nombre d'alertes récentes ;
- le score moyen des alertes ;
- le score maximal détecté.

Ces indicateurs permettent d'évaluer rapidement le niveau de risque actuel ainsi que l'intensité des activités frauduleuses détectées par le système.

La deuxième partie de la page contient un tableau interactif affichant les dernières alertes enregistrées. Le tableau présente plusieurs informations importantes :

- l'identifiant de l'alerte ;
- l'agence concernée ;
- l'identifiant de la transaction ;
- le score de fraude ;
- le seuil de déclenchement ;
- la catégorie marchande ;
- le timestamp de l'alerte.

Le tableau intègre également une mise en forme conditionnelle permettant de colorer les lignes selon le niveau de criticité du score :

- rouge pour les alertes critiques ;
- orange pour les alertes moyennes ;
- jaune pour les alertes faibles.

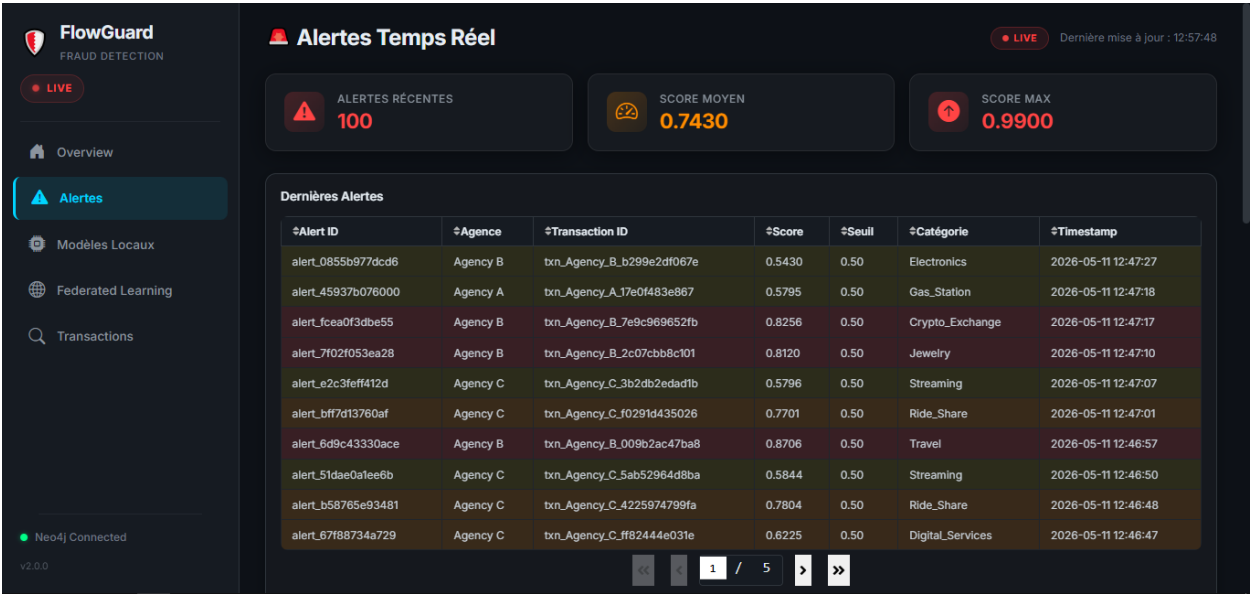


Figure 19 : Alerts_1

La troisième partie de la page contient plusieurs visualisations permettant d'analyser les alertes de fraude en temps réel. Le premier graphique est un graphique en aires empilées représentant l'évolution du nombre d'alertes par minute sur la dernière heure pour les agences A, B et C. Cette visualisation permet d'observer les variations du flux d'alertes et d'identifier rapidement les périodes d'activité suspecte élevée.

Le second graphique est un histogramme représentant la distribution des scores de fraude des alertes pour chaque agence. L'axe horizontal correspond aux scores de fraude compris entre 0 et 1, tandis que l'axe vertical représente la fréquence des alertes. Cette représentation permet d'analyser la répartition des niveaux de risque détectés par les modèles ML.

Enfin, la dernière visualisation est un diagramme en anneau représentant les catégories marchandes les plus associées aux alertes de fraude. Chaque segment correspond à une catégorie spécifique et affiche sa proportion dans l'ensemble des alertes enregistrées. Ce graphique permet d'identifier rapidement les secteurs d'activité les plus exposés aux comportements frauduleux.

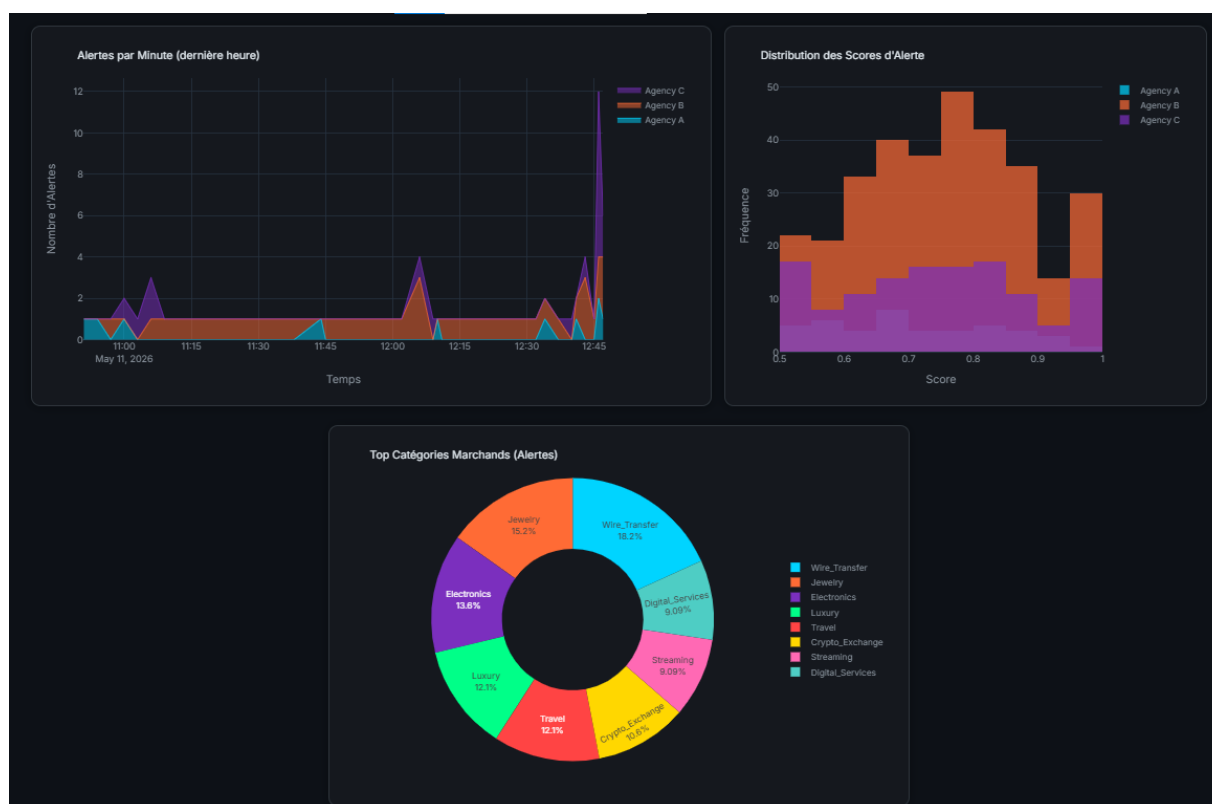


Figure 20 : Alerts_2

Page Modèles Locaux — Modèles ML Locaux

La page **Modèles ML Locaux** est dédiée au suivi des performances des modèles de Machine Learning utilisés par chaque agence. Elle permet d'analyser l'évolution des modèles locaux A, B et C ainsi que leurs métriques d'apprentissage.

La première partie de l'interface contient trois cartes représentant les agences A, B et C. Chaque carte affiche le numéro de version du modèle ainsi que plusieurs métriques importantes sous forme de barres de progression :

- Accuracy ;
- Precision ;
- Recall ;
- F1 Score.

Les cartes affichent également plusieurs informations complémentaires :

- le taux de fraude (Fraud Rate) ;
- le nombre d'échantillons d'entraînement (Train Samples) ;
- le seuil de classification (Threshold) ;
- la date du dernier entraînement (Last Trained).

La deuxième partie de la page contient un sélecteur de métrique permettant de choisir la métrique à analyser (Accuracy, Precision, Recall ou F1 Score). En dessous, un graphique en courbes affiche l'évolution de cette métrique au fil des différentes versions des modèles locaux. Chaque agence possède une courbe distincte avec sa propre couleur, ce qui permet de suivre facilement la progression des performances des modèles au cours du temps.

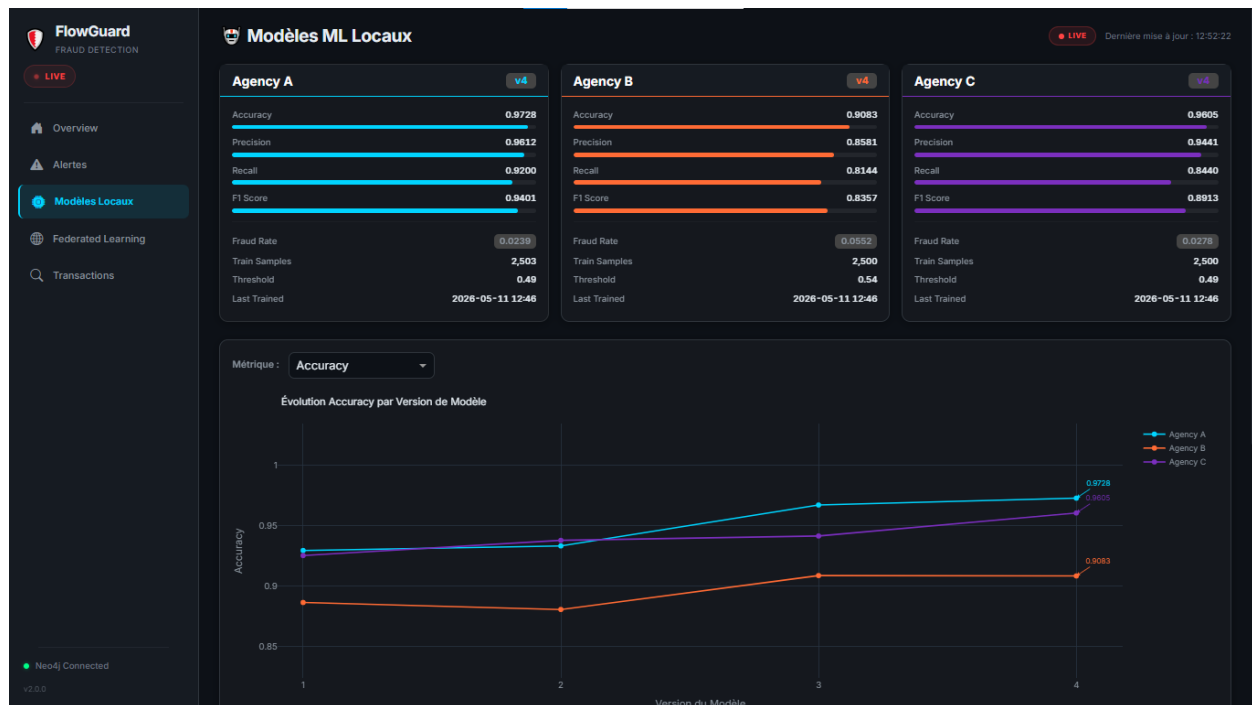


Figure 21 : ML_1

La dernière partie de la page contient deux composants d'analyse. Le premier est un diagramme en barres groupées permettant de comparer les performances des trois agences sur les différentes métriques ML. Cette visualisation facilite l'identification des modèles les plus performants selon chaque indicateur.

Le second composant est un tableau présentant l'historique complet des modèles locaux. Celui-ci affiche l'agence, la version du modèle, les métriques principales, le taux de fraude, le nombre d'échantillons d'entraînement ainsi que la date du dernier entraînement. Le tableau supporte le tri et la pagination afin de faciliter l'exploration des différentes versions des modèles.

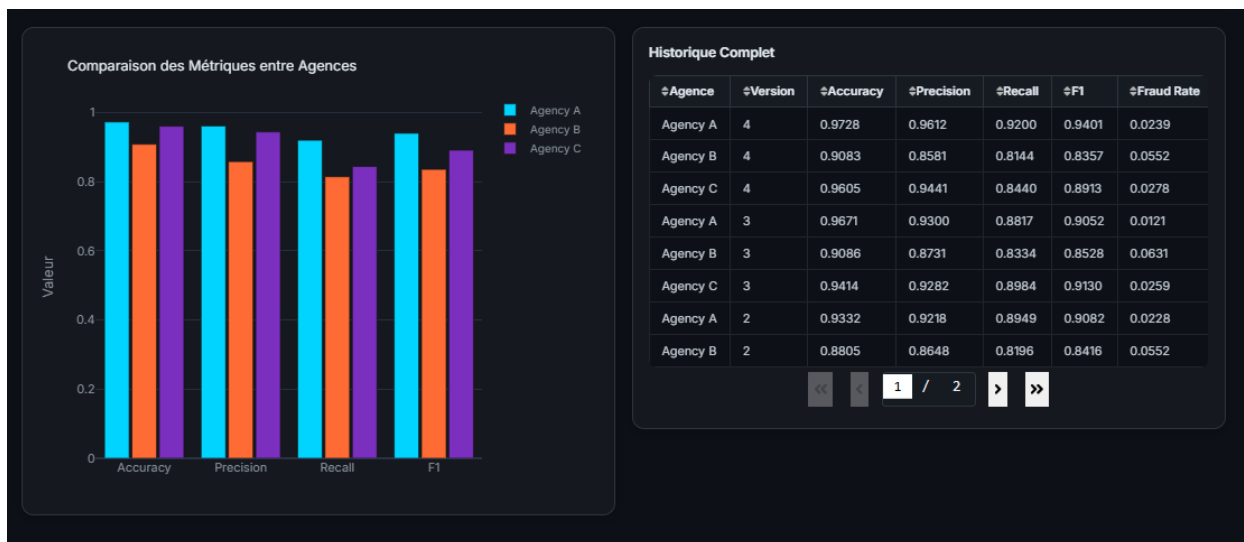


Figure 22 : ML_2

Page Federated Learning — Suivi des rounds fédérés

La page **Federated Learning** permet de superviser les différents rounds d'agrégation réalisés par le module FL de FlowGuard. Elle offre une vue détaillée de l'évolution du modèle global construit à partir des modèles locaux des agences A, B et C.

La première partie de la page contient une carte récapitulative du dernier round de Federated Learning. Cette carte affiche :

- l'identifiant du round (Round ID) ;
- la version globale du modèle ;
- la date et l'heure du round ;
- les agences ayant participé à l'agrégation.

À droite de cette carte, plusieurs badges présentent les principales métriques du modèle global :

- Accuracy ;
- F1 Score ;
- Recall ;
- Precision.

La deuxième partie de l'interface contient deux graphiques d'analyse. Le premier est un graphique en courbes représentant la progression des métriques du modèle global au fil des différents rounds FL. Chaque courbe correspond à une métrique spécifique et permet de suivre l'amélioration progressive des performances du modèle fédéré.

Le second graphique est un diagramme radar comparant le modèle global FL avec les modèles locaux des agences. Les axes du radar représentent :

- Accuracy ;
- Precision ;
- Recall ;
- F1 Score ;
- (1 - Fraud Rate).

Cette visualisation permet de comparer rapidement les forces et faiblesses du modèle global par rapport aux modèles locaux.

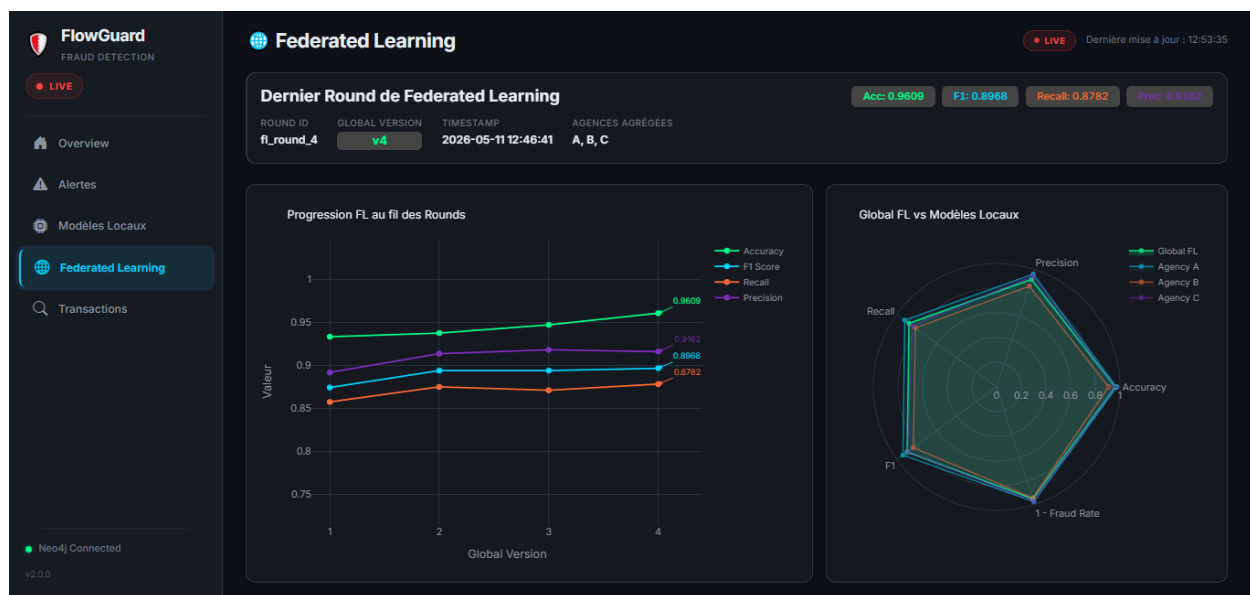


Figure 23 : FL_1

La dernière partie de la page contient un tableau présentant l'historique des rounds de Federated Learning. Le tableau affiche notamment :

- l'identifiant du round ;

- la version globale ;
- la date ;
- les agences participantes ;
- les métriques du modèle global ;
- le taux de fraude ;
- le chemin de l’artefact sauvegardé.

Le tableau supporte également la pagination et le tri des colonnes afin de faciliter l’analyse des différentes versions du modèle global.

Round ID	Global Version	Date	Agences	Accuracy	F1	Recall	Precision	Fraud Rate	Artifact
fl_round_4	4	2026-05-11 12:46	A, B, C	0.9609	0.8968	0.8782	0.9162	0.0465	cts/global/global_model_v4.pkl...
fl_round_3	3	2026-05-11 10:15	A, B, C	0.9473	0.8940	0.8711	0.9182	0.0379	cts/global/global_model_v3.pkl...
fl_round_2	2	2026-05-11 09:15	A, B, C	0.9378	0.8940	0.8750	0.9138	0.0384	cts/global/global_model_v2.pkl...
fl_round_1	1	2026-05-11 08:15	A, B, C	0.9335	0.8743	0.8575	0.8918	0.0362	cts/global/global_model_v1.pkl...

Figure 24 : FL_2

Page Transactions — Analyse des transactions

La page **Analyse Transactions** permet d’explorer et d’analyser en détail les transactions traitées par FlowGuard. Elle fournit une interface de filtrage et de consultation temps réel des opérations enregistrées dans Neo4j.

La première partie de la page contient une zone de filtres permettant de sélectionner dynamiquement les transactions à afficher. Plusieurs critères sont disponibles :

- l’agence ;
- le statut de fraude ;
- les transactions en ligne ;
- les transactions étrangères ;
- une plage de montants.

Un bouton **Appliquer** permet ensuite de lancer le filtrage et de mettre à jour les données affichées dans l’interface.

La deuxième partie contient un tableau interactif présentant les transactions correspondant aux filtres sélectionnés. Le tableau affiche plusieurs informations détaillées :

- l’identifiant de la transaction ;
- l’agence ;
- le montant ;
- la catégorie marchande ;
- la localisation ;
- l’heure ;
- le statut en ligne ;
- le statut étranger ;
- la prédiction du modèle ;
- le statut final de fraude ;
- le timestamp.

Les transactions suspectes sont mises en évidence grâce à une coloration spécifique des lignes afin de faciliter leur identification. Le tableau prend également en charge la pagination, le tri des colonnes ainsi que l’export des données affichées.

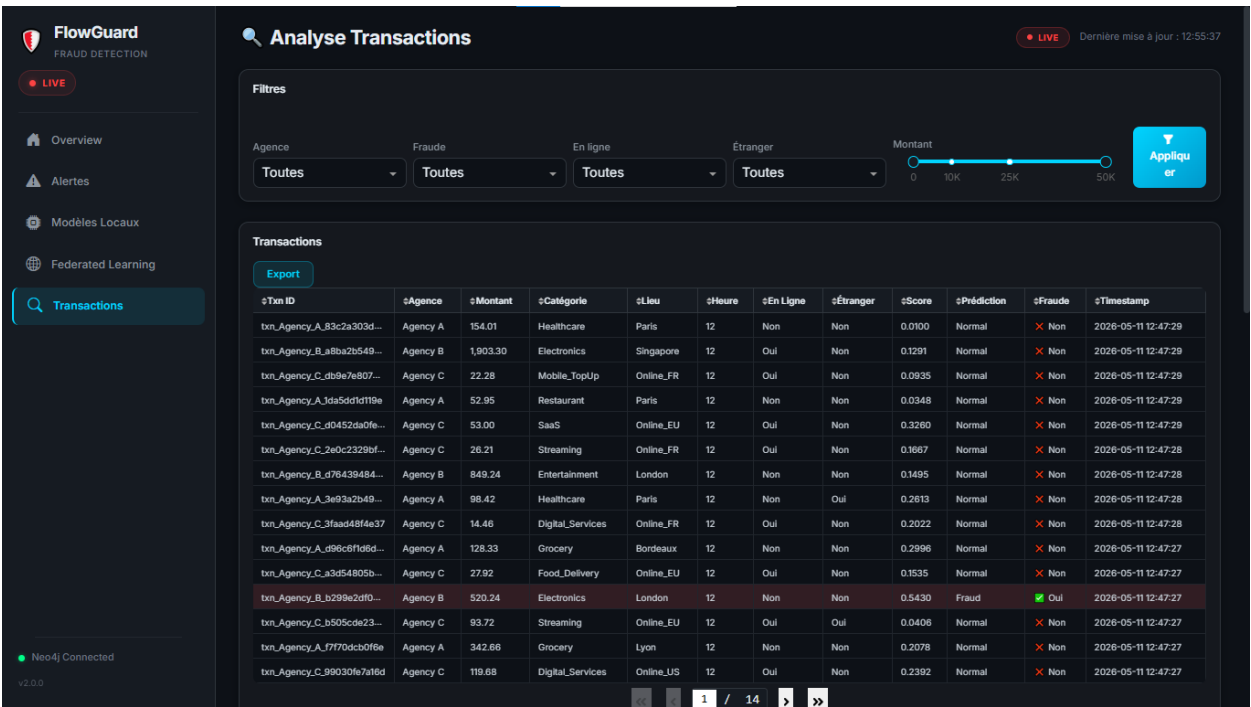


Figure 25 : Transactions_1

La deuxième partie de la page contient deux visualisations permettant d'analyser les comportements transactionnels du système. Le premier graphique est une heatmap représentant la fréquence des transactions selon l'heure et le jour de la semaine. Les couleurs indiquent l'intensité de l'activité transactionnelle, ce qui permet d'identifier rapidement les périodes de forte activité ou les comportements inhabituels.

Le second graphique est un diagramme de type boxplot représentant la distribution des montants des transactions par agence et par statut (Normal ou Fraud). Cette visualisation permet de comparer les montants des transactions normales et frauduleuses pour chaque agence, tout en mettant en évidence les écarts, les dispersions et les valeurs extrêmes détectées par le système.

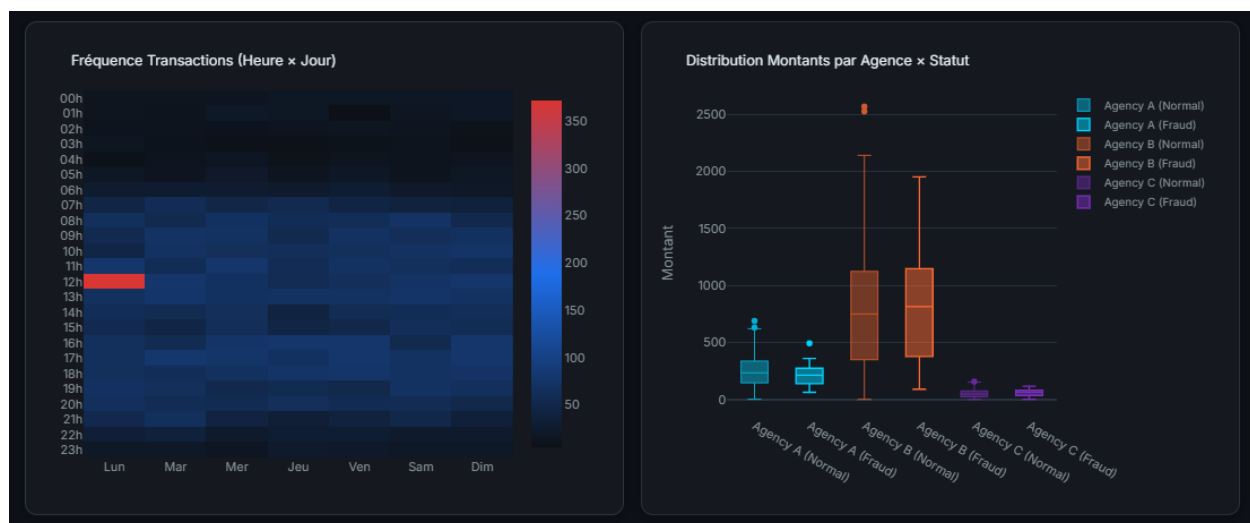


Figure 26 : Transactions_2

La dernière section de la page contient deux graphiques d'analyse avancée des transactions et des comportements de fraude. Le premier graphique présente les dix catégories marchandes les plus fréquentes sous forme de diagramme en barres horizontales. Une courbe supplémentaire affiche le taux de fraude associé à chaque catégorie, ce qui permet d'identifier les secteurs présentant le plus grand risque de fraude tout en comparant leur volume de transactions.

Le second graphique est un diagramme hiérarchique de type *Sunburst* représentant la relation entre les agences, le statut des transactions (Normal ou Fraud) et les catégories marchandes. Cette visualisation permet d'explorer la répartition des

transactions de manière hiérarchique et d'identifier rapidement quelles catégories et quelles agences génèrent le plus de transactions frauduleuses.

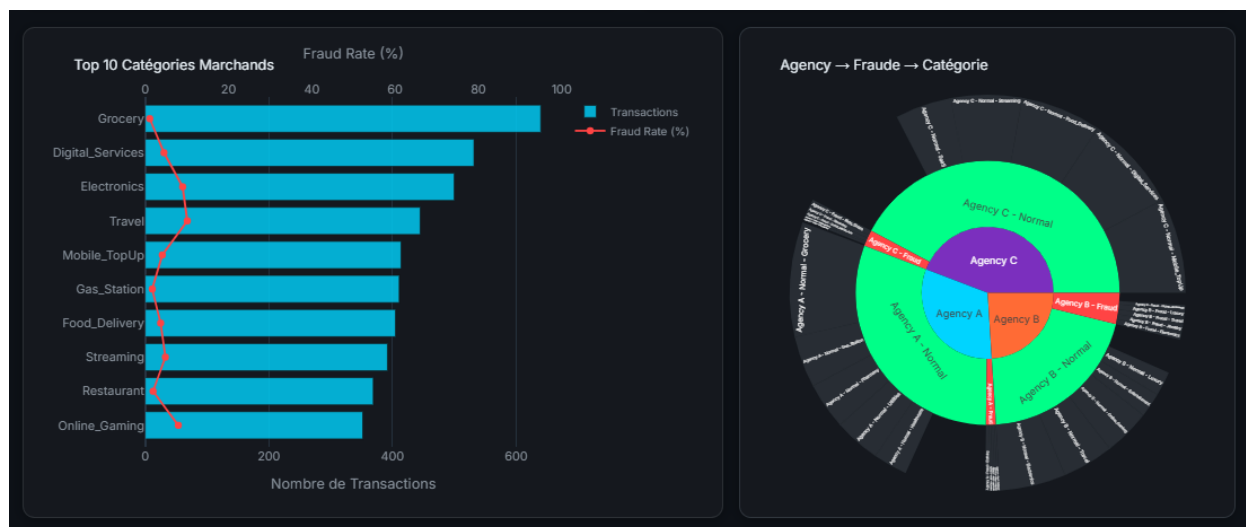


Figure 27 : Transactions_3

Conclusion du chapitre

Dans ce chapitre, nous avons présenté les principaux outils technologiques utilisés pour la conception et le développement du système FlowGuard. Les technologies choisies ont permis de construire une architecture temps réel capable de traiter les transactions financières, d'effectuer des prédictions de fraude et d'assurer un apprentissage distribué via le Federated Learning.

Les outils de streaming, de Machine Learning, de stockage graphe et de visualisation ont été intégrés afin de réaliser une solution complète de détection de fraude. La partie suivante du chapitre présentera les différentes étapes de réalisation du projet ainsi que les interfaces développées pour la supervision du système.

Conclusion et perspectives

Le développement des services financiers numériques et l'augmentation continue des transactions électroniques rendent les systèmes de détection de fraude de plus en plus importants. Les institutions financières doivent aujourd'hui être capables de traiter des flux massifs de données en temps réel tout en détectant rapidement les comportements suspects. Cependant, les approches classiques présentent plusieurs limites, notamment en matière de scalabilité, de rapidité de traitement et de confidentialité des données.

Dans ce contexte, le projet FlowGuard a proposé une solution complète de détection de fraude temps réel basée sur les technologies Big Data, le Machine Learning et le Federated Learning. L'objectif principal du projet était de concevoir une architecture distribuée capable de :

- générer et traiter des transactions financières en continu ;
- détecter automatiquement les transactions suspectes ;
- entraîner des modèles de Machine Learning locaux ;
- agréger les modèles via Federated Learning sans partager les données sensibles ;
- stocker et visualiser les informations en temps réel.

Pour atteindre ces objectifs, plusieurs technologies ont été intégrées dans une architecture unique. Apache Kafka a été utilisé comme plateforme de streaming afin d'assurer la communication entre les différents composants du système. Les transactions générées par les streams des agences sont nettoyées puis analysées par un pipeline ML utilisant des modèles supervisés basés sur SGDClassifier. Le système réalise ensuite un réentraînement périodique des modèles locaux avant d'effectuer une agrégation fédérée grâce à l'algorithme FedAvg.

Le projet a également intégré Neo4j afin de stocker les transactions, les alertes, les modèles et les rounds de Federated Learning sous forme de graphe relationnel. Cette base de données a permis de faciliter les analyses et l'alimentation du dashboard temps réel développé avec Dash et Plotly.

Les résultats obtenus montrent que l'architecture FlowGuard est capable de :

- traiter des transactions en temps réel ;
- détecter automatiquement des comportements frauduleux ;
- suivre l'évolution des performances des modèles locaux ;
- construire un modèle global via Federated Learning ;
- visualiser les informations grâce à une interface interactive moderne.

Le dashboard réalisé permet notamment de superviser :

- les alertes de fraude ;
- les flux transactionnels ;
- les performances des modèles ML ;
- les rounds de Federated Learning ;
- les analyses statistiques des transactions.

Le projet présente également plusieurs apports importants. Il combine dans une seule plateforme plusieurs concepts avancés tels que le streaming temps réel, le Machine Learning distribué, le Federated Learning et les bases de données graphes. Il permet aussi de simuler un environnement proche d'un système financier réel grâce à des comportements transactionnels variés et des scénarios de fraude réalistes.

Malgré ces résultats, certaines limites restent présentes. Les transactions utilisées dans le système sont simulées et non issues de données bancaires réelles. Les modèles de Machine Learning utilisés restent relativement simples par rapport aux architectures avancées de Deep Learning utilisées dans les systèmes industriels. De plus, le système de Federated Learning ne prend pas encore en compte certains mécanismes avancés comme :

- l'agrégation sécurisée ;
- la protection différentielle ;
- la gestion des clients malveillants ;
- l'apprentissage asynchrone.

Enfin, plusieurs perspectives d'amélioration peuvent être envisagées pour faire évoluer le projet :

- intégrer des modèles de Deep Learning pour améliorer la précision de détection ;
- utiliser des données financières réelles ou des datasets publics spécialisés ;
- déployer le système sur une infrastructure cloud distribuée ;
- ajouter des mécanismes de cybersécurité et de chiffrement des échanges ;
- intégrer des algorithmes avancés de Federated Learning ;
- développer des fonctionnalités d'analyse prédictive et d'explication des décisions des modèles ;
- connecter le système à des API financières réelles afin de traiter des transactions réelles en direct.

Ainsi, le projet FlowGuard constitue une base solide pour la conception d'un système intelligent de détection de fraude financière combinant traitement temps réel, apprentissage automatique distribué et visualisation interactive.

Webographie

PlantUML – Documentation officielle :

<https://plantuml.com/>

Python – Documentation officielle :

<https://www.python.org/>

Apache Kafka – Documentation officielle :

<https://kafka.apache.org/>

Scikit-learn – Documentation officielle :

<https://scikit-learn.org/>

Neo4j – Documentation officielle :

<https://neo4j.com/>

Dash – Documentation officielle :

<https://dash.plotly.com/>

Plotly – Documentation officielle :

<https://plotly.com/>

NumPy – Documentation officielle :

<https://numpy.org/>

Dash Bootstrap Components – Documentation officielle :

<https://www.dash-bootstrap-components.com/>

Cypher Query Language – Documentation Neo4j :

<https://neo4j.com/docs/cypher-manual/current/>